agent-harness — Manual

agent-harness — Manual

agent-harness is a Claude Code plugin that sets up, checks and maintains a **repository harness** for AI coding agents. The harness is the set of files, commands and hooks in your repo that tell an agent what the rules are, how to prove its work, and what it may not touch.

This manual is for:

- **Developers** adopting the plugin on a repo. You know git and your stack; you don't need to know this plugin yet.
- **Team leads** deciding how far to take it (the target level), what to commit for the whole team, and what to review.

Contents

File	Read it when
01-concepts.md	You want to know what a harness is, the five control layers, and how the maturity level is measured
02-file-reference.md	You want to know what a file in your repo does, who writes it, who reads it, and whether you may edit it
03-user-guide.md	You are installing the plugin, running setup, using the everyday skills, or something is blocking you
04-workspace.md	One feature spans several repos (backend, portal, app) checked out side by side

Reading order. Team leads: 01 → 03 (install, init, audit) → 02 as needed. Developers: the quick start below → 03 → 02 when a file surprises you. Read 04 only if you work across repos.

Quick start (5 minutes)

1. **Install the plugin** in the repo (project scope, so the whole team gets the same hooks):

   ```
   claude plugin marketplace add salescode-ai/agent-harness
   claude plugin install agent-harness@agent-harness --scope project
   ```

2. **Run setup** from a Claude Code session in the repo root:

   ```
   /agent-harness:harness-init
   ```

   It scans the repo without asking anything, shows you a file tree of what it will write, and waits for one "yes". Then it asks a few questions per layer, each with a default (reply `defaults` to accept them all). Use `/agent-harness:harness-init one-shot` to skip the questions entirely.

3. **Wire this checkout's git hooks** once setup is finished (the agent's guard allows only `git config core.hooksPath .githooks`, so the agent may also do this step):

   ```
   make setup
   ```

4. **Work as usual.** From now on:

   - every session starts with a short orientation (branch, active plan, warnings);

- dangerous commands (force-push, `git reset --hard`, `--no-verify`, …) are blocked;
- when the agent tries to stop with changed code, `check-fast` runs and blocks the stop if it fails;
- before saying "done", the agent runs the `harness-verify` skill (the gate plus independent reviewers).

5. **Check the result** any time:

   ```
   /agent-harness:harness-audit
   ```

   It reports a measured level such as `Level: Functional · target: Robust · blocked by: …` and a ranked list of fixes.

The three commands you type

Command	What it does
`/agent-harness:harness-init`	Set up, resume, or upgrade the harness in this repo. Argument `one-shot` skips the interview.
`/agent-harness:harness-audit`	Read-only audit. `quick` (default, monthly) or `full` (quarterly or after an incident).
`/agent-harness:harness-workspace`	In a folder of sibling repos: write workspace-level files only.

Everything else (verify, session, learn, garden, glossary, and the reviewer agents) is used by the agent itself or on request. See 03-user-guide.md.

Source of truth

This manual describes the plugin in `plugins/agent-harness/`. When the manual and the code disagree, the code wins: the skills (`skills/*/SKILL.md`), agents (`agents/*.md`), hook scripts (`scripts/`) and templates (`skills/harness-init/assets/`). The design rationale is in `docs/specs/2026-10-26-harness-v2-design.md`.

1. Concepts

What a repository harness is

An AI coding agent is a model plus everything around it: the instructions it loads, the commands it can run, the checks that catch its mistakes, and the limits on what it may touch. The design follows the common industry harness model:

> **Agent = Model + Harness**

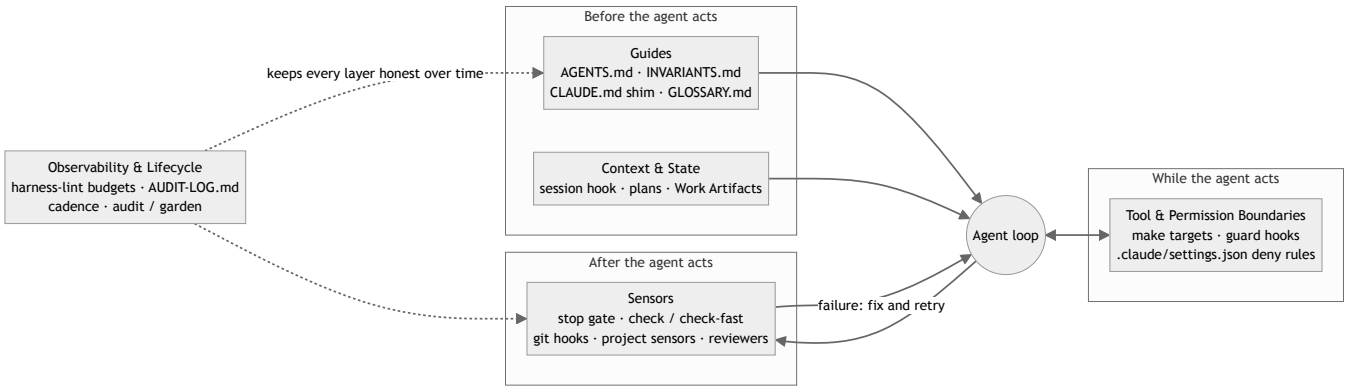
You don't control the model. You control the harness. So when an agent gets something wrong, the fix goes into the harness: a rule, a check, a better pointer. It does not go into a cleverer prompt you have to remember to paste next time. The `harness-learn` skill turns this into a habit. A repeated mistake becomes a sensor, a hook, an invariant or a footgun, preferring the strongest one that fits.

This plugin builds an **engineering-time** harness. That means files and checks in your repo that shape how agents write code. It does **not** cover runtime concerns such as tool dispatch, retries, cost ceilings or runtime metrics.

Principles the plugin follows everywhere:

- **Evidence or a marker.** Every path, command or env var the plugin writes comes from a `file:line` it saw. If it has no evidence, it writes `[verify]` or `TODO` and never invents one.
- **Ask only what code can't answer.** Anything the scan proves is applied automatically. Judgement calls are asked, and each one comes with a default.
- **Ratchet.** A rule earns its place by tracing to a real failure, incident or hard constraint. Generic advice doesn't count.
- **A model grading itself is not a measurement.** Levels and scores come from a script (`level.py`), not from the agent's opinion.
- **No named packs.** The harness works alongside any skill pack and never names one. The harness holds the facts; packs bring process.

The five control layers



Every scan finding, generated file, audit item and interview question is tagged with one of these layers. In files and scripts the layer keys are `guides`, `sensors`, `context`, `tools`, `observability`.

Guides (`guides`)

Controls	What the agent reads before acting: the map of the repo, the hard rules, vocabulary, decisions, and who may do what.
Acts	Before. Loaded at session start through the `CLAUDE.md` shim (`@AGENTS.md`, `@INVARIANTS.md`).
Prevents	The agent guessing: reading random files first, inventing commands, ignoring constraints nobody wrote down, renaming domain concepts, pushing to main because no one said not to.
Implemented by	`AGENTS.md` (Verification, Working Principles, Agent skills, Work Artifacts, Context loading, Agent Authorization, Modules, Footguns), `INVARIANTS.md`, `CLAUDE.md`, module `AGENTS.md`, `GLOSSARY.md`, `docs/ARCHITECTURE.md`, `docs/DECISIONS.md` + `docs/adr/`, `docs/agents/*.md`. Guarded by `guard-files.sh`, which keeps `CLAUDE.md` a shim and asks before `INVARIANTS.md` edits.
Example	`INVARIANTS.md` says `- No real customer or production data in seeds or fixtures — Severity: Critical — Enforced by: sensor no-prod-data`. An agent asked to "add realistic seed data" reads this first and uses synthetic data.

Sensors (`sensors`)

Controls	Checks that observe the work and fail loudly: the `check` / `check-fast` gate, project sensors for specific invariants, and the reviewer agents.
Acts	After, and during the loop: the stop gate runs when the agent tries to finish; git hooks run on commit and push; `harness-verify` runs before "done".
Prevents	"Done" claims with no evidence, broken builds handed to humans, rules that exist only on paper, and verification theatre (tests skipped or weakened to pass).
Implemented by	`harness.mk` targets `check`, `check-fast`, `lint`, `typecheck`, `test`; `stop-gate.sh` (Stop hook; SubagentStop marks a red implementer branch in its own worktree, which the Bash guard then refuses to merge); `.githooks/pre-commit` (check-fast) and `.githooks/pre-push` (check); `scripts/harness/sensors/` + `registry.json` + `fixtures/`; `sensor-proof.sh`; `red-proof.sh` (the change's tests fail without it, run by the verifier; a plan closes only with its `Red:` line); `gate-proof.sh` (a planted failing test turns `test T=` and check-fast red, run by init and upgrade; a gate never proven red is a sensors gap); agents `verifier`, `invariants-reviewer`, `scope-guard`, `cross-repo-reviewer`, `sensor-smith`; optional CI workflow `harness.yml`.
Example	The agent edits a file and tries to stop. `stop-gate.sh` sees uncommitted source changes, runs `check-fast`, it fails, and the stop is blocked with `Stop blocked by agent-harness: a repo's check-fast failed on your changes.` and a summary of the output: lines with failure markers (`FAIL`, `ERROR`, `expected`, `Tests run:…Failures`…), then the last 10 lines.

Invariants carry a severity:

Severity	Effect
Critical	Fails `check` and blocks the stop gate (once a machine enforces it).
Important	A warning; `harness-verify` stops and hands it to a human (`NEEDS-HUMAN`).
Nice-to-have	Appears in the audit only.

A sensor is wired in only after `sensor-smith` proves it. The proof checks that clean code passes, that a planted violation in a fixture fails with a message, and that clean code passes again. A sensor that checks only new code reads the added lines from `scripts/harness/sensors/lib/diffscan.py` instead of parsing `git diff` itself. Its proof also plants the violation after an added `++` line and runs it with an unresolvable `HARNESS_BASE`, which must fail. `harness-lint` re-runs every fixture (`sensor-proof.sh --recheck`), so a sensor that silently stops firing is caught.

Context & State (`context`)

Controls	What the agent loads for a given task, and where work in progress lives so it survives `/clear`, compaction or a new session.
Acts	Before (session orientation, Context loading table) and during (the plan is updated as steps finish).
Prevents	Lost work after a context reset, re-doing finished steps, carrying invented facts forward, plans and specs scattered across pack-specific folders, and an auto-loaded set so big it crowds out the task.
Implemented by	`session-context.sh` (SessionStart hook: branch, uncommitted count, active plan, next unchecked step, last progress line); `AGENTS.md` Context loading and Work Artifacts; `docs/plans/` (`active/`, `done/`, `TEMPLATE.md`); `docs/specs/`; `harness-session` skill; `.gitignore` entries for `.harness/`, `.worktrees/` and pack scratch folders; `lint.sh` size budgets.
Example	After `/clear`, the session starts with `[agent-harness] active plan: docs/plans/active/2026-10-20-export.md` and `[agent-harness] next unchecked: 3. Add CSV writer → verify: make test T=…`. The agent picks up at step 3.

Tool & Permission Boundaries (`tools`)

Controls	Which commands the agent runs (one command surface), how a fresh machine or worktree gets a working dev setup, and what the agent may never do.
Acts	During: every Bash command and every file write passes a guard hook.
Prevents	Raw tool commands that differ per developer, destructive git operations, bypassing hooks, writing secrets, overwriting `CLAUDE.md` from the shell, seeding a non-local database, and "works on my machine" setups.
Implemented by	`Makefile` + `harness.mk` (or `.agents/harness.json` for another runner); `scripts/harness/cmd.sh` (command resolver); `guard-bash.sh`/`guard_bash.py` and `guard-files.sh` (PreToolUse hooks); `.claude/settings.json` deny rules; `scripts/harness/setup.sh`, `setup-proof.sh`, `local-guard.sh`, `dev-check.sh` (`make dev-check`: start, check, always stop); `docs/SETUP.md`; agent `setup-smith`.
Example	The agent runs `git commit --no-verify -m "wip"`. The guard denies it: `Bypassing git hooks (--no-verify / -n) is blocked by INVARIANTS.md. Fix the failing check instead.`

Observability & Lifecycle (`observability`)

Controls	The health of the harness itself over time: size budgets, audit cadence, and the measured history.
Acts	After, and on a schedule: `harness-lint` in every `check`, the session hook's staleness warning, monthly/quarterly audits, weekly gardening.
Prevents	Harness rot: dead links, stale commands, an `AGENTS.md` that grew into a manual, rules duplicated in three places, sensors that no longer fire, and levels that are claimed but not real.
Implemented by	`lint.sh` (budgets, links, ADR index, command names, invariant format, sensor recheck); `AGENTS.md` `## Cadence`; `docs/harness/AUDIT-LOG.md` (one script-measured line per audit); `level.py`; skills `harness-audit` and `harness-garden`; agent `harness-auditor`.
Example	The last `AUDIT-LOG.md` row is 45 days old and Cadence says `quick audit: every 30 days`. The next session shows `[agent-harness] harness audit is 45 days old — run /harness-audit`.

Maturity levels

The level is **measured, never declared**. `level.py` evaluates fixed criteria against the repo and reports the highest level whose criteria all pass, plus its own criteria. Each level includes the criteria of the levels below it. If nothing passes, the level is `none`.

Level	Criteria checked by `level.py` (layer)
Experimental	`AGENTS.md` and `INVARIANTS.md` both exist (guides) · `CLAUDE.md` is absent or contains `@AGENTS.md` (guides) · `scripts/harness/lint.sh` passes (context)
Functional	`check` passes: with `--run-checks` it is resolved and run; without it, `.harness/last-check` must record a pass (sensors) · automatic enforcement: `.githooks/pre-commit` with `core.hooksPath=.githooks`, or a hook manager config (`lefthook.yml`, `.pre-commit-config.yaml`, `.husky/*`) that runs `check-fast`, or a CI file that runs `check` (sensors) · at least one invariant is machine-enforced: its `Enforced by:` names a make target, sensor, hook, guard, CI job, lint, test or a script path. Human review, an agent, `none` or `not yet enforced` does not count (guides)
Robust	`docs/plans/` exists (context) · `.claude/settings.json` has `permissions.deny` rules (tools) · at least one sensor in `scripts/harness/sensors/registry.json` and `sensor-proof.sh --recheck` passes (sensors) · no lint budget failure (context) · `AGENTS.md` has a Cadence section (observability) · no invariant under `## Security` or `## Data Integrity` is `not yet enforced` (guides)
Enterprise	a CI file (`.github/workflows/*.yml`, `.gitlab-ci.yml`, `.circleci/config.yml`, `Jenkinsfile`) runs `check` (sensors) · `.claude/settings.json` `enabledPlugins` pins `agent-harness` (tools) · `CODEOWNERS` (root, `.github/` or `docs/`) covers `AGENTS.md` and `INVARIANTS.md` (guides) · `AUDIT-LOG.md` has 2+ rows and the last level is not lower than the previous one (observability) · in a workspace, every member is at Functional or higher (observability)

Each layer gets a score: the share of its criteria that pass, across all four tiers. The audit shows these as `guides 0.80`, `tools 0.50` and so on.

Target vs current level

- **Current level:** what `level.py` measures now.
- **Target level:** what the team wants. It is chosen in the Guides section of `harness-init` and recorded in `INVARIANTS.md` as `- Harness target: <level>` (the template default is `Robust`).

The target decides the scope of setup and audit:

Target	Layers set up by `harness-init`
Functional	Guides and Sensors
Robust	All five
Enterprise	All five, plus CI, CODEOWNERS and audit history

`harness-audit` never recommends work beyond the target. It says in one line what the next level would need.

Why measured, not declared

A level the agent assigns itself is a self-grade. It drifts toward "looks good" and can't be compared over time. `level.py` reads files, runs lint, and (with `--run-checks`) runs the real `check`. The audit appends only these numbers to `docs/harness/AUDIT-LOG.md` (date, level, layer scores, health score, auto-loaded tokens). Judgements stay in the conversation. That gives the Enterprise "scores holding or improving" criterion something real to compare against.

Note: the level criteria are this plugin's own definitions.

2. File reference

This section covers every file and folder the harness creates or manages in a repo, and then the workspace files. For each one you get its layer, its purpose, who writes it, who reads it, and whether a human may edit it.

Writers: *init* = `harness-init` (copied from `skills/harness-init/assets/` or filled from the scan) · *human* = you · *script* = a harness script at run time · *agent* = a named plugin agent.

Layout at a glance

```
repo/
├── AGENTS.md  INVARIANTS.md  CLAUDE.md  GLOSSARY.md        [guides]
├── Makefile (+ include harness.mk)  harness.mk              [tools]
├── .agents/harness.json            (only for a non-make runner) [tools]
├── .agents/skills/                 (repo-specific skills)   [guides]
├── .claude/settings.json                                    [tools]
├── scripts/harness/  cmd.sh lint.sh setup.sh setup-proof.sh local-guard.sh sensor-proof.sh red-
proof.sh gate-proof.sh dev-check.sh
│   └── sensors/      registry.json  no-prod-data.sh  lib/diffscan.py  fixtures/   [sensors]
├── .githooks/        pre-commit  pre-push                    [sensors]
├── docs/  ARCHITECTURE.md TESTING.md SETUP.md DECISIONS.md adr/ plans/ specs/ agents/
harness/AUDIT-LOG.md
├── .github/workflows/harness.yml   (only after a CI yes)   [sensors]
├── <module>/AGENTS.md (+ CLAUDE.md) (only where it earns its place) [guides]
└── gitignored: .harness/  .worktrees/  .env.local  (+ pack scratch folders)
```

Guides files

`AGENTS.md` (root)

Layer	Purpose	Written by	Read by	Humans edit?
guides	The map every tool loads: where things are and which commands prove work. It points to detail and does not explain it. Budget ≤ 100 lines (warn), 120 (fail).	init (skeleton, Verification from passing commands, Module map from the scan); human fills the `<…>` and `TODO` parts	every agent via `CLAUDE.md`; `session-context.sh` (Cadence, plan folders); `lint.sh` (size, links, `make` names); `level.py` (Cadence); `verifier` (Verification)	Yes. It is yours. Keep it short.

Sections in the template:

Section	Holds	Notes
Header	"Read INVARIANTS.md first", one-line purpose, link to `docs/ARCHITECTURE.md`	
`## Verification`	Short form: done = `make check` (harness-verify first); fast loop `make check-fast`, `make test T=<path>`; all commands: `make help`. Never a list of every target. Another runner: its equivalents and its help command (`just --list`, `npm run`, `task --list`)	`lint.sh` fails if a `make <x>` named here has no target, or if the runner is not make and it names `make`

Section	Holds	Notes
`## Working Principles`	Think before coding · Simplicity first · Surgical changes · Goal-driven · Speak the domain	Pack guidance pasted into `CLAUDE.md` is moved here (duplicates dropped)
`## Agent skills`	Precedence (`INVARIANTS.md` > `AGENTS.md` > `.agents/skills/` > plugin skills > tool defaults), artifacts, proof, one process per phase, no writes to `CLAUDE.md`, pointers to `docs/agents/*`, team settings	The block skill packs must obey; `guard-files.sh` sends `CLAUDE.md` writers here
`## Work Artifacts`	Paths for specs, plans, ADRs, worktrees (`.worktrees/`), scratch (`.harness/`), pack working notes	`session-context.sh` reads plan folders from backticked `…plans/…` paths here
`## Context loading`	Table: task → files to read first	
`## Agent Authorization`	"May, without asking" / "Must stop and ask"	Always asked in init; never defaulted in one-shot
`## Modules`	One line per module `AGENTS.md`	
`## Conventions`	Commit/branch conventions link; `CLAUDE.md` is `@`-only	
`## Cadence`	`quick audit: every 30 days · full audit: every 90 days or after an incident · garden: every 7 days`	The session hook parses `quick audit: every N days`; `level.py` requires the word `Cadence` for Robust
`## Footguns`	`**<name>** — what goes wrong. What to do instead. See docs/adr/NNNN-slug.md.`	

Module `AGENTS.md` (+ `CLAUDE.md`)

Layer	Purpose	Written by	Read by	Humans edit?
guides	Local rules for a folder that handles auth, payments, PII, migrations or a public API, has its own toolchain, or has ~10+ lines of its own rules. Budget ≤ 80 lines (warn). Its `CLAUDE.md` contains only `@AGENTS.md`.	init (only for `risk_paths` modules or a separate toolchain)	agents working in that folder; `invariants-reviewer` (footguns); `lint.sh` (size, links, TODOs; it skips gitignored copies and build dirs such as `target/`, `build/`, `dist/`, `out/`, `.gradle/`, `node_modules/`)	Yes

`INVARIANTS.md`

Layer	Purpose	Written by	Read by	Humans edit?
guides (rules), sensors (enforcement)	Hard rules. Breaking one needs explicit human sign-off; no skill or prompt overrides it.	init (Harness Integrity block verbatim, target, no-prod-data rule; new rules only after you approve them)	every agent; `invariants-reviewer`; `lint.sh` (format, budgets); `level.py` (target, enforcement, security/data rules)	Yes. The agent's edits trigger an **ask** from `guard-files.sh`.

Format of every bullet (enforced by `lint.sh`, except under the two budget sections):

```
- <rule> — Severity: <Critical|Important|Nice-to-have> — Enforced by: <make target | hook | CI job | sensor <name> | human review | not yet enforced>
```

Special lines and sections:

Item	Meaning
`- Harness target: <Experimental|Functional|Robust|Enterprise>`	The target level; `level.py` reads it (default Robust)
`## Harness Integrity`	Shim-only `CLAUDE.md`, size limits, ADR index, skills layout, no weakening tests, never bypass hooks/CI
`## Security`, `## Data Integrity`	For Robust, nothing here may be `not yet enforced`
`## API Contract`, `## Testing`, `## Performance`	Placeholders; fill only with real constraints
`## Context Budget`	`- auto-loaded tokens: warn 3000, fail 5000`
`## File Size Budget`	`- AGENTS.md: warn 100, fail 120` · `- module AGENTS.md: warn 80, fail 999999` · `- SKILL.md: warn 120, fail 999999`
`not yet enforced (<n> errors, <date>)`	Debt for a tool wired in changed-files mode
`not yet enforced (declined <date>)`	The repo has no such tool (e.g. no linter); keeps the gap visible

`CLAUDE.md` (shim)

Layer	Purpose	Written by	Read by	Humans edit?
guides	Claude Code's entry point. It contains only `@`-references, `#` headings, blank lines and HTML comments.	init (existing content moved to `AGENTS.md`)	Claude Code at session start; `lint.sh`; `level.py`	Only to add `@` lines. Agents get denied by `guard-files.sh` (Edit/Write) and `guard_bash.py` (`>`, `>>`, `tee`, `cp`/`mv`/`install`, `curl -o`, `sed -i`/`perl -i`).

```
# Claude Code entry point — instructions live in the files below

@AGENTS.md
@INVARIANTS.md
```

`GLOSSARY.md`

Layer	Purpose	Written by	Read by	Humans edit?
guides	Domain vocabulary: Term · Meaning · Avoid · Code name. For multiple contexts, use `GLOSSARY-MAP.md` → one glossary per context.	init (header only unless you define terms); `harness-glossary` (rows after you say what they mean)	agents (Working Principle 5); `invariants-reviewer`; `harness-glossary`; `harness-garden` (candidates)	Yes. Definitions come from people, never from code.

`docs/ARCHITECTURE.md`, `docs/DECISIONS.md`, `docs/adr/`

File	Layer	Purpose	Written by	Read by	Humans edit?
`docs/ARCHITECTURE.md`	guides	Components, data flow, boundaries (Mermaid)	init (skeleton)	agents on design tasks; `harness-auditor`	Yes
`docs/DECISIONS.md`	guides	Index, one row per ADR: ADR · Decision · Status · Date · Ticket	init; `harness-garden` adds missing rows	`lint.sh` (every ADR listed, every link exists)	Yes
`docs/adr/NNNN-slug.md`	guides	One decision per file (`TEMPLATE.md`, `0001-record-decisions.md` shipped)	human; `harness-learn`, `harness-session`, `harness-glossary` propose	`invariants-reviewer`; `lint.sh`	Yes

`docs/agents/*`

File	Purpose
`issue-tracker.md`	How agents use the tracker. One variant is copied: `github` or `gitlab` (detected from `git remote`), or `jira` / `local` (asked). The Jira/GitHub/GitLab variants describe approval-gated status updates: drafted progress or evidence comments, posted only after a yes.
`triage-labels.md`	Label roles: `needs-triage`, `needs-info`, `ready-for-agent`, `ready-for-human`, …
`domain.md`	What to read before exploring: glossary, `DECISIONS.md`, ADRs

Layer guides; written by init; read by agents and skill packs via `AGENTS.md` → Agent skills pointers. Humans may edit them (fill tracker key and labels).

`.agents/skills/`

Repo-specific skills (`<name>/SKILL.md` with `name:` and `description:` front matter). `.claude/skills/<name>` may hold **only symlinks** to them. Layer guides; written by humans (or by `install-local.sh` for non-Claude tools); checked by `lint.sh` (section 5) and audit question 10.

Context & State files

File	Purpose	Written by	Read by	Humans edit?
`docs/plans/TEMPLATE.md`	Plan skeleton: Status · `Ticket:` (Jira key, `#123`, `.scratch/...` or `none (<reason>)`; `lint.sh` fails on a placeholder in `active/`) · Goal · Acceptance criteria (each proved by a test/command) · Steps (`→ verify:`) · Progress log (append-only) · Decisions. An `**Evidence**:` line points to the local `.harness/evidence/<plan file name>` where harness-verify appends `### <date> · <commit> · <verdict>` blocks (never committed; shared via the PR description and tracker comment)	init	`harness-session`	Yes

File	Purpose	Written by	Read by	Humans edit?
`docs/plans/active/`	Plans in progress	humans and agents (`harness-session`)	`session-context.sh` (matches the ticket key in the branch name, then the `Ticket:` line, then the branch slug, else the newest with unchecked items, ≤ 14 days old; prints the last verification entry); `verifier` (criteria)	Yes
`docs/plans/done/`	Finished plans (moved only when `plan-check.sh fresh` passes on the local evidence, including its `Red:` line)	`harness-session`, `harness-garden`	humans	Yes
`docs/specs/`	Designs and specs (`YYYY-MM-DD-<topic>.md`)	humans / skill packs	agents	Yes

Tool & Permission files

`Makefile` and `harness.mk`

Layer	Purpose	Written by	Read by	Humans edit?
tools	One command surface for people, agents, hooks and CI. `harness.mk` is included from the root `Makefile` **after** your own default target.	init copies `harness.mk`, adds the include and the `HARNESS_*` variables	`cmd.sh` (via `make -n <target>`); stop gate; git hooks; `verifier`; CI; `lint.sh`	`Makefile`: yes. `harness.mk`: leave the plugin's copy alone and configure it through variables.

Targets in `harness.mk`:

Target	Runs	Always defined?
`help`	Lists harness targets plus `##`-commented targets of your Makefile (default goal)	yes
`check`	`lint typecheck test harness-lint`, then `check: OK`	yes
`check-fast`	`lint typecheck` + `$(HARNESS_TEST_CHANGED) $(CHANGED)` if set (`{CHANGED}` in the value: substituted there, not appended), else `$(HARNESS_TEST_ALL)` when `HARNESS_TEST_SECS` ≤ 60; target < 60 s	yes
`lint` / `typecheck`	`$(HARNESS_LINT)` / `$(HARNESS_TYPECHECK)`	yes
`test`	`$(HARNESS_TEST_ALL)`, or `$(HARNESS_TEST_ONE) $(T)` with `T=<path>` (`{T}` in the value: substituted there, not appended, e.g. `-Dtest='{T}'`). A value must keep the test's exit code: end in `&& :`, never `; :` (harness-lint warns), and fail when 0 tests ran (layout.md)	yes
`smoke`	`$(HARNESS_SMOKE)` (default prints `smoke: no runnable surface`, which setup reports as `smoke n/a`)	yes
`harness-lint`	`bash scripts/harness/lint.sh`	yes
`setup`	`$(HARNESS_SETUP) = bash scripts/harness/setup.sh`	yes

Target	Runs	Always defined?
`seed` / `reset`	`local-guard.sh`, then `$(HARNESS_SEED)` / `$(HARNESS_RESET)`	only when the variable is set **above** the include
`dev`	`$(HARNESS_DEV)`	only when set above the include
`dev-check`	`dev-check.sh start`: `make dev` in the background (own process group, log in `.harness/dev-check.log`), waits up to `HARNESS_HEALTH_SECS` (120) for `HARNESS_HEALTH` (a URL answering 2xx/3xx, or a command exiting 0), runs `RUN='<cmd>'` if given, then always stops it. Exit: `RUN`'s status, 1 when never healthy	only when `HARNESS_DEV` and `HARNESS_HEALTH` are set
`dev-stop`	`dev-check.sh stop`: stops the server `dev-check` started from this checkout	same as `dev-check`

Variables: `HARNESS_LINT`, `HARNESS_TYPECHECK`, `HARNESS_TEST_ALL`, `HARNESS_TEST_ONE`, `HARNESS_TEST_CHANGED` (usually `bash scripts/harness/related-tests.sh <stack> "<runner>"`), `HARNESS_TEST_SECS` (full-suite baseline seconds), `HARNESS_SMOKE`, `HARNESS_SETUP`, `HARNESS_SEED`, `HARNESS_RESET`, `HARNESS_DEV`, `HARNESS_HEALTH` + `HARNESS_HEALTH_SECS` (enable `dev-check`), `HARNESS_DEV_BLOCK` (run scripts the Bash guard refuses besides `make dev`, e.g. `run-local.sh spring-boot:run`), `HARNESS_SETUP_SMOKE` (`0` = setup does not run smoke), `HARNESS_TEST_CHANGED_CONFIRMED` (`1` = a custom `HARNESS_TEST_CHANGED` was reviewed; lint stops warning), `HARNESS_BASE` (base for `CHANGED`, which also lists untracked new files; default `origin/HEAD`, else `origin/main`, else `origin/master`). To keep a target your Makefile already has, set `HARNESS_DEFINE_<NAME>=0` above the include (`HARNESS_DEFINE_HELP=0`, `HARNESS_DEFINE_CHECK_FAST=0`, …).

Example root `Makefile` after init:

```
build: ## Build the app
    ./gradlew --console=plain build

HARNESS_BASE      = origin/master
HARNESS_LINT      = @echo "lint: none (declined 2026-10-26)"
HARNESS_TEST_ALL  = ./gradlew --console=plain test
HARNESS_TEST_ONE  = ./gradlew --console=plain test --tests
HARNESS_SEED      = psql "$$DATABASE_URL" -f db/seed.sql
include harness.mk
```

Changed-files mode for a tool that already fails on main:

```
UI_CHANGED   = $(call harness_changed,ui/,ts tsx)
HARNESS_LINT = @if [ -n "$(strip $(UI_CHANGED))" ]; then cd ui && npx eslint $(UI_CHANGED); else
echo "lint: no changed UI files"; fi
```

Repo-specific setup steps live in your `Makefile` (or `.agents/harness.json`) as `setup-deps`, `setup-services`, `setup-migrate`; `setup.sh` runs each one only if it resolves. `setup-smith` wires them. Your `setup-services` does not inherit the `*_PORT` values setup.sh picked as environment variables (Compose would let them override `--env-file`); it reads `.env.local`, and `setup-migrate` and `seed` get the ports re-read from `.env.local` after it.

`.agents/harness.json`

Layer	Purpose	Written by	Read by	Humans edit?
tools	Registers a runner other than make. Each command is printed verbatim; `{T}` (unquoted) is replaced by the shell-quoted test path.	init (only if you pick another runner)	`cmd.sh` first, before make; `lint.sh` (AGENTS.md must name the same runner and commands)	Yes

```
{ "runner": "just",
  "commands": { "check": "just check", "check-fast": "just quick", "test": "just test {T}",
                "setup": "just bootstrap", "seed": "just seed", "reset": "just reset",
                "smoke": "just smoke", "dev": "just dev" },
  "help": "just --list" }
```

`help` is optional: the runner's list command, named in `AGENTS.md` Verification ("All commands: `just --list`") and returned by `cmd.sh <dir> help` when `commands.help` is absent.

`.claude/settings.json`

Layer	Purpose	Written by	Read by	Humans edit?
tools	Team-shared: pins the plugin (`extraKnownMarketplaces`, `enabledPlugins`) and adds permission **deny** rules on top of the guard hooks	init after you approve the rules (Claude Code may refuse the write; then you add it yourself from the shown JSON)	Claude Code; `level.py` (deny rules → Robust, pinned plugin → Enterprise)	Yes. Review it like code.

Proposed deny rules: secrets files, `Read(./**/secrets/**)`, `Read(./**/*.pem)`, `Read(./**/*.key)`, plus dangerous infra/production commands found in the repo (e.g. `Bash(terraform apply:*)`), each with its `file:line`. Allow rules for the runner (e.g. `Bash(make check:*)`). The `.env` rules deny `.env`, `.env.local` and real environment files, while `.env.example` / `.env.sample` stay readable.

`scripts/harness/*`

All are copied by init (or `install-local.sh`) and run from the repo root. Leave them alone unless you are fixing a bug. Changes belong upstream in the plugin: after a plugin update, `harness-init` refreshes unmodified copies (user guide → Updating).

File	Layer	Purpose	Read/run by
`cmd.sh`	tools	Resolves a target to a command: `.agents/harness.json` first, else `make -s -C <dir> <target>` if `make -n <target>` succeeds. Usage `cmd.sh <dir> <target> [T]`. Exit 3 = not wired, 4 = make missing (both are **not** a pass).	git hooks, `setup.sh`, `setup-proof.sh`, skills, `verifier`, `harness-auditor`
`lint.sh`	observability	`harness-lint`: core files exist; `CLAUDE.md` shim; size and token budgets; ADR index; skills layout; relative links; `make`/runner names in AGENTS.md exist; `INVARIANTS.md` format; TODO/`[verify]` warnings; plan `Ticket:` through `plan-check.sh ticket` (fail; links, URLs and key lists accepted); changed source files check-fast tests nothing for (warn; source = extensions of the stacks whose build files the repo has, `scripts/harness/` excluded); sensor recheck; `.gitignore` covers `.harness/`. Prints `harness-lint: OK (≈N auto-loaded tokens)` or `FAIL` lines. Tokens ≈ UTF-8 bytes / 4, the same estimate `level.py` reports.	`make check`, `level.py`, init, garden, learn

File	Layer	Purpose	Read/run by
`setup.sh`	tools	`make setup`: prereq versions (`.nvmrc`, `.python-version`, `.sdkmanrc`, `.tool-versions`), registry sign-in hints, `.env` from `.env.example` if missing, `.env.local` with worktree values, then `setup-deps`, services (`docker compose … up -d`), `setup-migrate`, `seed`, git hooks, `smoke`. The maven `settings.xml` hint appears only when `setup-deps` did not pass. Smoke runs only when nothing is listed under "needs you" and `HARNESS_SETUP_SMOKE` is not `0`; the unset default reports `smoke n/a`. Never `sudo`, never fetches secrets.	`make setup`, `setup-proof.sh`, workspace `ws.sh setup`
`setup-proof.sh`	tools	Clones committed HEAD to a temp folder, resolves and runs `setup` there under its own compose project, tears down. Prints `setup-proof: OK`, `FAIL (<step>)` or `SKIPPED (docker not available)`. `HARNESS_PROOF_KEEP_ENV=1` copies `.env.local` in.	`setup-smith`, init step 5, `harness-audit full`
`sensor-proof.sh`	sensors	`sensor-proof.sh <name>` (3-step proof) or `--recheck` (fixture fires, clean passes, for all sensors). A `"scope": "diff"` sensor must also fire when each fixture file starts with an added `++` line (step 2b) and fail loudly with an unresolvable `HARNESS_BASE` (step 2c)	`sensor-smith`, `lint.sh`, `level.py`, audit
`red-proof.sh`	sensors	`red-proof.sh [<base>]` (default: merge-base with `origin/HEAD`, else `origin/main`/`origin/master`, else `HEAD`). In a scratch worktree holding the current code, each changed test file runs with `test T=<file>` and must pass (green control), then must fail once the changed production files are put back to `<base>` (red). Last line and exit: `red: proven (…)` 0 · `red: n/a (no production code changed …)` 0 · `red: not proven (…)` 1 (no test changed, or every changed test passes without the change) · `red: inconclusive (…)` 2 (tests don't pass in the scratch worktree, or `test` not wired). Shares `node_modules`, `.venv`/`venv`, `.env`, `.env.local` by symlink.	`verifier` (harness-verify), whose `Red:` evidence line `plan-check.sh fresh` requires
`gate-proof.sh`	sensors	`gate-proof.sh [junit5|pytest|vitest|jest]` (default: from the root build file, then the test files). In a scratch worktree holding the current code, plants `HarnessGateProofTest.java` / `test_harness_gate_proof.py` or `harness-gate-proof.test.<ext>` beside an existing test: `test T=<it>` and check-fast (with `HARNESS_BASE=HEAD`, lint and typecheck off) must pass while it passes, then fail once it fails; `test T=<no such test>` must fail. Writes `gate_red_proven` (+ `gate_red_reason`) to `.harness/init-state.json`. Last line and exit: `gate: proven red (…)` 0 · `gate: NOT proven (…)` 1 · `gate: inconclusive (…)` 2 (no supported stack or test, target not wired, or a passing control failed). check-fast is `n/a` when it runs no tests by design.	init step 5, upgrade step 3, `harness-audit full`; `level.py` reads the result
`dev-check.sh`	tools	`dev-check.sh start <health> [secs] [run-cmd]` / `stop`: behind `make dev-check` / `make dev-stop` (see the target table). The pid of the dev process group is in `.harness/dev.pid`.	`make dev-check`, `make dev-stop`, agents checking a running app

File	Layer	Purpose	Read/run by
`local-guard.sh`	tools	Refuses `seed`/`reset` unless every DB-like host is local. Checks env, `.env` and `.env.local` (`DATABASE_URL`, DB-scheme `*_URL`, `DB_HOST`, `*_DB_HOST`, `PGHOST`, `MYSQL_HOST`, `MONGO_URL`, `REDIS_URL`) and Spring `spring.datasource.url` in `src/main/resources/application.properties`	yml and the application-local*, -dev*, -test* profiles, plus every profile named in SPRING_PROFILES_ACTIVE (env, .env, .env.local): an active prod profile is read and its non-local hosts refused. Inactive profiles are not read. Oracle thin URLs (jdbc:oracle:thin:@host:1521/X, @//host`) are parsed for their host.
`related-tests.sh`	sensors	`HARNESS_TEST_CHANGED` helper: `related-tests.sh [-C <module>] <stack> "<runner>" <changed files>` runs the tests a change can break. jest/vitest: their related-tests lookup; go: each changed package; maven/gradle: `Foo` → `FooTest`/`FooTests`/`FooIT`/`TestFoo` plus tests naming `Foo`; pytest: `test_foo.py`/`foo_test.py`, tests naming `foo`, else tests naming its package; node: `foo.test.js` plus tests naming `foo`. Nothing related → `tests: no …`, exit 0	`check-fast` (`HARNESS_TEST_CHANGED`)
`plan-check.sh`	context	One reading of plans shared by `lint.sh`, `ws.sh lint` and the skills. `ticket <plan>`: exit 0 valid (Jira key anywhere in the value, also as a link or list; `#123`; an issue/browse URL; `.scratch/…`; `none (<reason>)`), 1 missing or placeholder, 2 a key outside the Jira project in `docs/agents/issue-tracker.md`. `fresh <.harness/evidence/<plan>>`: exit 0 when the latest local entry is PASS, its commit has the same code as HEAD (plans excluded) and no code is uncommitted. `fresh` also needs a `Red: proven (…)` or `Red: n/a (<reason>)` line in the latest PASS entry (2.0.8+).	`lint.sh`, `ws.sh lint`, `harness-session` Close
`assets.json`	tools	Manifest of the plugin-owned copies: `{"plugin": "<version>", "sha256": {"<path>": "<sha256>"}}` for `scripts/harness/*` (not `sensors/registry.json` or fixtures), `.githooks/*` and `harness.mk`. A copy whose hash matches is unmodified and may be refreshed; any other content is `modified` and left alone. No marker lines are added to the files themselves. Written by `assets.py install`/`refresh`; commit it. `plugin` is the plugin version the harness was last installed or upgraded with (`harness-upgrade` reads it as the recorded version).	`assets.py`, `migrate.py`, session hook, `harness-audit`
`sensors/registry.json`	sensors	`{"sensors": [{"name","severity","invariant","run","mode","clean","fixture","dest","scope"?}]}`. `mode: paths` substitutes `{paths}`; `mode: copy` copies fixture files into `dest`, runs, then removes them. `"scope": "diff"` marks a sensor that checks only new code (mode copy).	`sensor-proof.sh`, `level.py`

File	Layer	Purpose	Read/run by
`sensors/no-prod-data.sh`	sensors	`no-prod-data.sh <path…>`: flags real-looking emails (except `@example.*`, `.test`, `.invalid`, `.local`), `prod`/`production` in host or URL contexts only (after `@` or `//`, or as a `host`/`*_HOST`/`url` value; prose and comments are ignored), `rds.amazonaws.com` hosts, real-looking phone numbers, and `.sql`/`.dump` files > 1 MB. Output `file:line: reason — use synthetic data (INVARIANTS.md Data Integrity)`.	`make check` once wired, `sensor-proof.sh`
`sensors/lib/diffscan.py`	sensors	Plugin-owned. The lines new code adds since the merge-base with `HARNESS_BASE` (commits, staged, unstaged, untracked; not `sensors/fixtures/`): `added_lines(base)` → `{path: {lineno: text}}`, or as a command `path:lineno:text` lines. Hunk-aware (an added `++ x` line is content, not a header). Exits 2 with `sensor: cannot resolve HARNESS_BASE <ref>; fetch it or set HARNESS_BASE` when the base or merge-base can't be resolved.	new-code sensors, `sensor-smith`
`sensors/fixtures/<name>/`	sensors	Planted violations, e.g. `no-prod-data/seed.sql` with `jane@acme.com`. Never edit them to make a proof pass.	`sensor-proof.sh`

Local-host rules in `local-guard.sh`: `localhost`, `::1`, `host.docker.internal`, `*.local`, `*.test`, `*.localhost`, `127.x.x.x` (digits only), and single-label names that are a service in a compose file in the repo (`docker-compose*.yml`, `compose*.yml`) are local. Any other single-label name (e.g. `prod`) is refused with `<host> is not a localhost name or a service in this repo's compose files`. Every host in a multi-host URL is checked.

`.env`, `.env.local`, `compose.harness.yml`

File	Purpose	Written by	Humans edit?
`.env`	Real local values. Created from `.env.example` by `setup.sh` only if missing; never overwritten.	`setup.sh` (first copy), then you	Yes. Agents are denied by `guard-files.sh`.
`.env.example` / `.env.sample`	Variable names and safe defaults; `*_PORT` and `DB_NAME` here drive worktree isolation	you	Yes. Agents may write these.
`.env.local`	Gitignored, per-checkout values: `COMPOSE_PROJECT_NAME`, `*_PORT` (main checkout: the `.env.example` port unless something other than this compose project holds it), `DB_NAME=<db>_<project>`. Your other lines are kept on re-run. `HARNESS_SHARED_SERVICES=1` here is honoured.	`setup.sh`	Yes (managed keys are rewritten)
`compose.harness.yml`	Local-only services when the repo has no compose file	`setup-smith`, **only after an explicit yes** (never by "defaults" or one-shot)	Yes

`docs/SETUP.md`

Layer tools. The dependency inventory: one row per external dependency (Dependency · Kind · Version · Source/evidence · Local mode `service|stub|shared-env|not needed` · Details from). Resolved rows cite `file:line`; unresolved rows record where you said to read about them. Written by init and `setup-smith`; read by developers and `harness-auditor`. Humans may edit it.

Sensors files

File	Purpose	Written by	Read by	Humans edit?
`.githooks/pre-commit`	Runs resolved `check-fast`. Prints `harness: setup in progress, skipping` while `.harness/init-state.json` lacks `"gate_wired": true`; skips with a one-line message (`harness: check-fast not wired, skipping` / `harness: scripts/harness/cmd.sh missing, skipping`) when the target isn't wired or `cmd.sh` is missing. Before that, a staged plan close (a plan added under `docs/plans/done/` or `plans/done/`, or a `Status: done` line) runs `plan-check.sh fresh .harness/evidence/<plan file name>` and blocks with its message unless the evidence is a fresh PASS (`HARNESS_ALLOW_UNVERIFIED_DONE=1` skips only when `.harness/evidence/` does not exist).	init / `install-local.sh`	git (via `core.hooksPath .githooks`, set by `make setup`) or your hook manager	Prefer not
`.githooks/pre-push`	Same, with `check`	same	same	Prefer not
Hook manager config	husky: `bash .githooks/pre-commit # harness check-fast` in `.husky/pre-commit`; lefthook: `harness-check-fast` / `harness-check` commands; pre-commit: `repo: local` hooks `harness-check-fast`, `harness-check` (`stages: [pre-push]`)	init (Sensors layer)	the manager	Yes
`docs/TESTING.md`	Test layout, what must be tested, fixtures and mocks, "a test proves a change only if it fails without it"	init (skeleton)	agents writing tests	Yes
`.github/workflows/harness.yml`	CI: `make harness-lint`, then `make check` on PRs and pushes to main; weekly lint-only run	init, **only after a CI yes**	CI; `level.py` (Enterprise)	Yes (add toolchain setup)

Observability files

File	Purpose	Written by	Read by	Humans edit?
`docs/harness/AUDIT-LOG.md`	One line per audit: `| date | level | g=… s=… c=… t=… o=… | health=N | tokens=N |`	only `level.py --log` (init final report, `harness-audit`); a row identical to the last one is not appended	`session-context.sh` (age vs Cadence), `level.py` (Enterprise trend), `harness-audit full` (compare)	**No.** Script-measured only.

Gitignored state

`assets/gitignore.append` adds `.harness/`, `.worktrees/`, `.env.local`. Init also ignores pack scratch folders the scan found.

Path	Purpose	Written by	Read by
`.harness/scan.json`	Scan result keyed to `head`: `stacks`, `modules`, `commands[{module,role,cmd,source,baseline}]`, `versions`, `services`, `env_vars`, `migrations`, `seeds`, `hook_manager`, `ci`, `harness_files`, `pasted_guidance`, `pack_folders`, `risk_paths`, `is_worktree`	`scan.py --write` (init, `repo-cartographer`); init adds `baseline`	init (resume), audit, garden, `sensor-smith`, `setup-smith`
`.harness/init-state.json`	`{"layers_done":[…], "answers":{…}, "gate_wired":bool, "gate_reason":"…", "target":"…"}`. Written with `"gate_wired": false` **before** any harness file; set `true` only when `check-fast` resolved and passed.	`harness-init`	init (resume); stop gate and git hooks (silent until `gate_wired` is true); session hook ("setup in progress" while a layer in scope for `target` is missing: Experimental/Functional need `guides`, `sensors`; Robust/Enterprise or no target need all five; warns once if the file is unreadable; "gate not wired — <gate_reason>" once all layers are done but `gate_wired` is not true)
`.harness/last-check`	`<fingerprint> pass <date>` after a green stop-gate run	`stop-gate.sh`	`level.py` without `--run-checks`
`.harness/last-green-repo`	Fingerprint of the last green tree, so an unchanged tree isn't re-checked	`stop-gate.sh`	`stop-gate.sh`
`.harness/last-fail-repo`, `.harness/stop-blocks`	Fingerprint of the last blocked tree and the count of blocks in a row (re-check bound, max 3)	`stop-gate.sh`	`stop-gate.sh`
`.git/harness-red/<branch>`	`<branch>\t<worktree>\t<date>`: a worktree subagent stopped with `check-fast` failing on this branch (in the shared git dir, so every worktree sees it)	`stop-gate.sh --subagent` (written on red, removed on green)	`guard_bash.py` (denies merge / cherry-pick / `pull .` of that branch)
`.worktrees/`	Git worktrees (Work Artifacts)	you / skill packs	`lint.sh` fails if it exists but isn't ignored

Delete `.harness/` freely except mid-init (you'd lose resume state).

Workspace files

These are written only in the workspace folder by `/agent-harness:harness-workspace`, never inside a member repo. Details are in 04-workspace.md.

File	Layer	Purpose	Written by	Read by	Humans edit?
`harness-workspace.json`	guides/tools	Manifest: `name`, `issueKey`, `repos[{name,path,remote,base,harness,owns}]`, `mergeOrder`	skill	`ws.sh`, session hook, stop gate, `level.py`, `cross-repo-reviewer`	Yes (agent edits trigger an ask)
`AGENTS.md`	guides	Routing table (repo → path → owns → its `AGENTS.md`), Verification, Cross-repo rules, Work Artifacts, Agent skills, Authorization; `## Member invariants` links when imports exceed the budget	skill; `ws.sh claude-md` appends links	agents	Yes
`CLAUDE.md`	guides	`@AGENTS.md`, `@INVARIANTS.md`, `@<repo>/INVARIANTS.md` per present member	`make claude-md`	Claude Code	No, regenerate it
`INVARIANTS.md`	guides	Contract first, merge order, no breaking API change without a version, no cross-repo imports, one branch name	skill	agents, `cross-repo-reviewer`	Yes
`Makefile` + `ws.sh`	tools	`status`, `changed`, `check`, `check-fast`, `setup`, `claude-md`, `test R= T=`, `branch B= R=`, `bootstrap`, `harness-lint`	skill	you, agents, `harness-verify`	Prefer not
`scripts/harness/cmd.sh`	tools	Resolver used by `ws.sh` for each member	skill	`ws.sh`	No
`scripts/harness/assets.json`	tools	sha256 of the workspace copies of `cmd.sh`, `plan-check.sh` and `ws.sh`, plus the recorded plugin version, so `harness-upgrade` can refresh them	`assets.py`	session hook	No
`.env.workspace`	tools	Local wiring, e.g. `API_BASE_URL=http://localhost:<port>` from the backend's `.env.local`	skill	you (copy into the member yourself)	Yes
`plans/`	context	`TEMPLATE.md`, `active/`, `done/` for cross-repo plans	skill	session hook, `harness-session`, `cross-repo-reviewer`	Yes
`.claude/settings.json`	tools	Pins the plugin; denies secrets reads across members	skill (or you, if the write is refused)	Claude Code	Yes
`.harness/`	—	Stop-gate state per member: `last-green-<name>`, `last-check-<name>`	`stop-gate.sh`	stop gate, `level.py`	—

3. User guide

3.1 Install

Claude Code (recommended)

Push the `agent-harness` repository somewhere your team can read it, then in each repo:

```
claude plugin marketplace add salescode-ai/agent-harness
claude plugin install agent-harness@agent-harness --scope project
```

Project scope writes to the committed `.claude/settings.json`, so everyone on the repo gets the same hooks and guards. `harness-init` also writes this pin (`enabledPlugins: {"agent-harness@agent-harness": true}`) when you approve the settings.

The plugin brings:

- **Skills:** `harness-init`, `harness-audit`, `harness-upgrade` and `harness-workspace` (you invoke these); `harness-improve` (offered at the end of init and audit; you or the agent start it), `harness-verify`, `harness-session`, `harness-learn`, `harness-garden` and `harness-glossary` (the agent uses these when they fit, or you ask for them).
- **Agents:** `repo-cartographer`, `verifier`, `invariants-reviewer`, `scope-guard`, `cross-repo-reviewer`, `harness-auditor`, `sensor-smith`, `setup-smith`.
- **Hooks:** SessionStart (`session-context.sh`), PreToolUse on Bash (`guard-bash.sh`), PreToolUse on Edit/Write/MultiEdit (`guard-files.sh`), Stop (`stop-gate.sh`), SubagentStop (`stop-gate.sh --subagent`).

Requirements: `git`, `bash`, `python3` (3.8+ syntax; macOS system Python 3.9 works) and `make` unless you register another runner. The guard hooks fail open when `python3` is missing: they allow the action and stay quiet.

Without the plugin installed

A harness has two halves:

1. **Committed repo files work for everyone, with any tool:** the guides (`AGENTS.md`, `INVARIANTS.md`, `CLAUDE.md`, `GLOSSARY.md`, `docs/`); `Makefile` + `harness.mk` + `scripts/harness/` (`make check`, `check-fast`, `test`, `setup`, `seed/reset` behind `local-guard.sh`, `harness-lint`, the sensor proofs); the `.githooks/` once `make setup` has run in that clone; CI if you opted in.
2. **Plugin only (Claude Code):** the hooks (stop gate, Bash and file guards, session orientation), the skills and the reviewer agents. None of these are copied into the repo.

Team rollout. Commit `.claude/settings.json` with `extraKnownMarketplaces` (`agent-harness` → `salescode-ai/agent-harness`) and `enabledPlugins: {"agent-harness@agent-harness": true}`, so every teammate's Claude Code offers the plugin when they trust the folder. `harness-init` asks before writing it; one-shot leaves it as a TODO; `harness-improve` reports "team rollout settings missing" as a gap until it is committed. For Codex, Cursor, Copilot or Gemini, `install-local.sh` copies the skills to `.agents/skills/`; those tools get no agent hooks, so the git hooks and CI do the enforcing.

Headless and CI runs (`claude -p`)

The skills read their own files through `${CLAUDE_PLUGIN_ROOT}` (`references/`, `assets/`, `scripts/scan.py`, `assets.py`, `level.py`), and that folder is outside the repo. Interactive sessions ask once; a non-interactive `claude -p` run cannot ask, so in the default mode init stops before writing anything ("outside the folders this session is allowed to touch"). Give the run the plugin folder and a mode that does not need prompts:

```
# installed plugin: its cache folder (holds every installed version)
claude -p "/agent-harness:harness-init one-shot" \
  --add-dir ~/.claude/plugins/cache/agent-harness \
  --permission-mode auto
# a local checkout instead of an installed plugin
claude -p "/agent-harness:harness-init one-shot" \
  --plugin-dir /path/to/agent-harness/plugins/agent-harness \
  --add-dir /path/to/agent-harness/plugins/agent-harness \
  --permission-mode auto
```

Instead of `--permission-mode auto`, you can keep the default mode and pre-approve only what the skills run: `--allowedTools "Read(~/.claude/plugins/cache/agent-harness/**)" "Bash(python3 *agent-harness*)" "Bash(bash *agent-harness*)" "Bash(make *)"`. The hooks (guards, stop gate) run the same way in headless sessions; `bypassPermissions` does not turn them off.

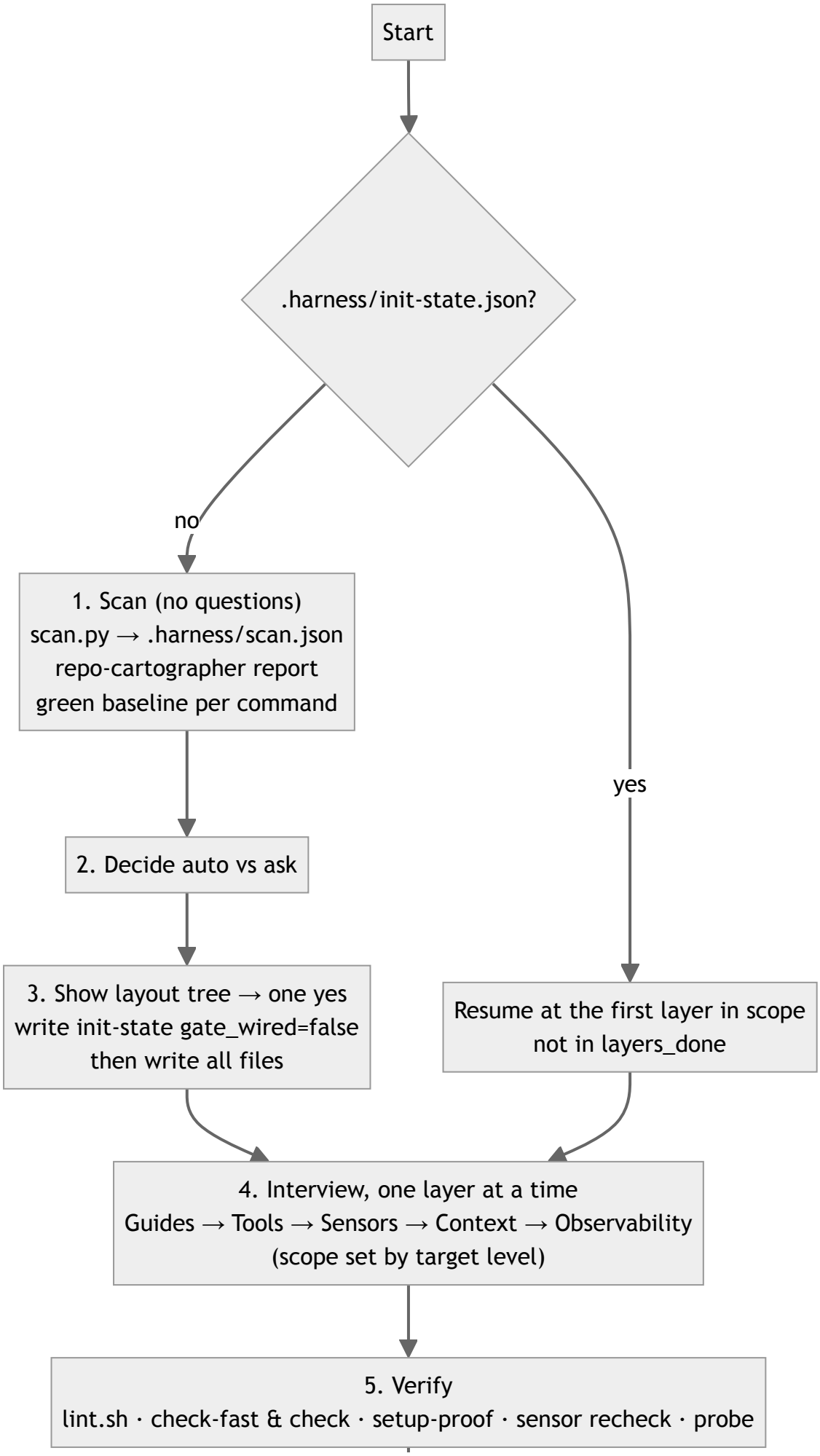
Codex, Cursor, Copilot, Gemini

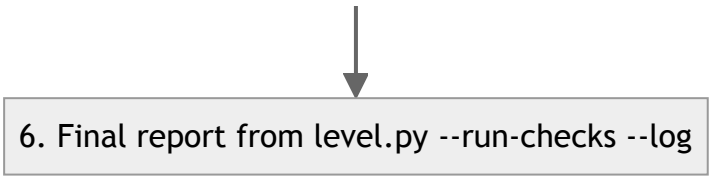
```
bash install-local.sh /path/to/repo
```

This copies the skills to `.agents/skills/`, the agent checklists to `.agents/agents/`, the harness scripts to `scripts/harness/`, and the git hooks to `.githooks/`. Existing files are skipped. Agent hooks (stop gate, guards, session context) are **Claude Code only**. With other tools, the git hooks, `make check` and optional CI are what enforce the gate. In tools without subagents, `harness-verify` runs each reviewer's checklist itself in a separate pass, reading `.agents/agents/<name>.md`. After installing, run `harness-init` (or fill the templates yourself) to reach Experimental. `install-local.sh` also makes `scan.py` and `level.py` available to the copied skills.

3.2 `/agent-harness:harness-init`

Run it once per repo, again to resume an unfinished setup, or to upgrade a v1 harness. While it runs it is the process: it does not start planning, test-first or review skills from other plugins.


Start
.harness/init-state.json?
no
yes
1. Scan (no questions)
scan.py → .harness/scan.json
repo-cartographer report
green baseline per command
2. Decide auto vs ask
3. Show layout tree → one yes
write init-state gate_wired=false
then write all files
Resume at the first layer in scope
not in layers_done
4. Interview, one layer at a time
Guides → Tools → Sensors → Context → Observability
(scope set by target level)
5. Verify
lint.sh · check-fast & check · setup-proof · sensor recheck · probe



Step 1: Scan

No questions in this step. It runs `scan.py . --write` to produce `.harness/scan.json`, dispatches `repo-cartographer` (which checks the scan and adds module boundaries, risk areas and pack residue, with `file:line` for each claim), and runs a **green baseline**: every detected command except install/dev/watch modes, on the untouched repo, with a 5-minute timeout each, non-interactive (`CI=true` for Jest/react-scripts, `-B` for Maven, `--console=plain` for Gradle). Each gets `baseline: pass`, `fail: <n> errors` or `skipped: <why>`. On a fresh repo it also runs the before/after **probe** prompt (`references/probe.md`).

Step 2: Auto vs ask

Applied automatically (evidence-backed): commands that pass the baseline; an existing compose file, seeds and git-hook manager; paths, dependency inventory and skeletons; `.gitignore` entries; moving pack guidance pasted into `CLAUDE.md` into `AGENTS.md`; catalog sensors that match evidence and pass their proof.

Always asked: target level; Agent Authorization; new invariants and severity; footguns and incidents; glossary definitions; adding a dependency or tool; CI opt-in; `.claude/settings.json` permissions; generating a compose file or seeds; where to read unresolved dependencies; isolated or shared worktree services; stub / shared environment / none for external services.

Step 3: Layout tree, then write

You see one tree with every file marked `auto-filled ✓`, `skeleton + TODO`, `needs your answer ?`, `~ extend` or `→ move`, tagged with its layer. Edits to your existing content are gathered into one diff. After your one yes:

1. `.harness/init-state.json` is saved with `"gate_wired": false` first, so the stop gate and git hooks stay silent while the harness is half-built.
2. Scripts are copied to `scripts/harness/`, hooks to `.githooks/`, templates to the repo. `harness.mk` is included after your default target, and `HARNESS_*` variables are set from commands that passed. `core.hooksPath` is **not** set at this point.
3. A tool whose baseline failed is wired in changed-files mode with a `not yet enforced (<n> errors, <date>)` invariant. A tool the repo doesn't have is **declined**: `HARNESS_LINT = @echo "lint: none (declined <date>)"` plus a `not yet enforced (declined <date>)` invariant.
4. `HARNESS_BASE` is written from `origin/HEAD`, else `origin/main`, else `origin/master`, else the upstream branch, and listed under *Assumed*.

Step 4: Interview, one layer per section

Each section looks like this:

```
## Sensors — checks that catch mistakes before a human does
Found:    eslint passes (package.json:12) · pytest fails on main (56 errors) · husky present
Will do:  pre-commit runs check-fast via husky · pytest wired in changed-files mode
1. Severity of "no prod data in seeds"?   a) Critical ➡️   b) Important  c) Nice-to-have
...
Reply with numbers and letters ("1a 3b"), or "defaults" to accept every ➡️.
Assumed — change any time: hook names · check-fast base origin/main · fixture folder
```

At most 5 questions per section, each with a ➡️ default. Low-stakes items are listed under *Assumed* and not asked.

Layer	Asks (defaults)
Guides	Target level (Functional / **Robust** / Enterprise, recommended from the scan) · Agent Authorization "May" and "Must stop and ask" · footguns (offered from fix/revert hotspots) · seed glossary terms (**later**)
Tools	Runner (**make** / register another) · services per worktree (**isolated** / shared) · generate `compose.harness.yml` (**show first**; needs an explicit yes) · each external service (**stub** when a contract exists / shared dev URL / not needed) · `.claude/settings.json` rules (**apply** / show only) · then "Where can I read the details for X?" per unresolved dependency. `setup-smith` wires and proves `make setup`.
Sensors	Severity per proposed invariant (**Important**) · catalog sensors (**build and prove**, via `sensor-smith`) · missing tool (**plain test or script** / add dependency); no linter at all (**declined** / add one) · git hooks (**on** / off) · CI workflow (**no** / yes, shown after the "where checks run" table)
Context & State	Context loading rows (**proposed**) · plan/spec paths (**`docs/plans/`**, **`docs/specs/`**) · pack working notes folder (**`.scratch/`**, gitignored)
Observability	Nothing by default; cadence and budgets are assumed. Enterprise with CI: add a CI audit job (**no**).

After each section, its files are written and `init-state.json` records the layer and the answers.

Step 5: Verify

1. `bash scripts/harness/lint.sh`. FAILs are fixed; TODO warnings are listed.
2. Resolve and run `check-fast`, then `check`. A target that doesn't resolve (exit 3/4) is **not** green. `gate_wired` becomes `true` only when `check-fast` resolved and exited 0. Then `gate-proof.sh` plants a failing test in a scratch worktree: `make test T=<it>`, `make test T=<no such test>` and `check-fast` must each exit non-zero (`gate_red_proven`). A test command ending in `; :`, or one that passes when 0 tests ran, fails this proof.
3. Setup proof in a fresh clone (when Tools is in scope). Commit the wiring first; the proof sees only committed files.
4. `bash scripts/harness/sensor-proof.sh --recheck`.
5. The probe again, and the difference is reported.

Step 6: Final report

```
Level: <measured> · target: <target> · blocked by: <first blockers>
Applied automatically: <n> — <one line each>
Assumed — change any time: <list>
Still open: <TODOs with file:line> · <invariants "not yet enforced">
Next: /harness-audit in a month · /harness-garden weekly
```

It runs `level.py . --json --run-checks --log`, which also writes the first `AUDIT-LOG.md` row.

Then **run `make setup` yourself** if you chose "git hooks: on" without a hook manager. A one-shot init never runs `make setup` (it changes git config and starts services): its report lists hooks as `pending make setup` and `make`

`setup` as the first Next step. It sets `core.hooksPath .githooks`. The agent may also run `git config core.hooksPath .githooks`: the guard allows exactly that value and denies any other.

Check your setup after init

Commit the init first: the setup proof uses committed code only. The commands below use `make`; with another runner, use the matching commands from `.agents/harness.json`.

1. **Measured level.** Run `/agent-harness:harness-audit` (read-only). ✅ `Level: <X> · target: <Y>`, a score per layer, and "blocked by" naming what is missing. Fix what it lists first.
2. **The commands work.** `make help`, `make check`, `make check-fast`, `make harness-lint`. ✅ All exit 0. A tool reported as `skipped` or `declined` is not enforced yet; `INVARIANTS.md` records it as `not yet enforced`.
3. **Git hooks are on.** `git config core.hooksPath` prints `.githooks` (or your husky / lefthook / pre-commit setup), and `.harness/init-state.json` has `"gate_wired": true`. If not, run `make setup`.
4. **The gates actually block** (two minutes, then revert):
   - Ask Claude to break one test and finish. ✅ The stop is blocked with `Stop blocked by agent-harness: … check-fast failed`.
   - `git commit` that broken change. ✅ pre-commit refuses it.
   - Ask Claude to run `git commit --no-verify` or `git push --force`. ✅ The guard denies both.
   - Revert with `git checkout -- <file>` (`git reset --hard` is blocked on purpose).
5. **Sensors and setup are proven.** `bash scripts/harness/sensor-proof.sh --recheck` (every sensor still catches its planted violation) and `bash scripts/harness/setup-proof.sh` (`make setup` works in a fresh clone). ✅ Every sensor proved; setup verdict OK.
6. **Session orientation.** Start a new Claude session in the repo. ✅ No warning about the gate not being wired, git hooks off, or scripts older than the plugin.
7. **Team knowledge.** `grep -n "TODO\|<" AGENTS.md INVARIANTS.md GLOSSARY.md docs/agents/issue-tracker.md` lists what init left for you (Agent Authorization, footguns, glossary terms, tracker site). Fill them, then run `/agent-harness:harness-improve`.

Steps 1–4 passing means the harness is configured and enforcing; 5–7 take it from working to solid.

One-shot, resume, upgrade

- **`/agent-harness:harness-init one-shot`** skips step 4 and takes every default. New files start with `<!-- generated in one pass: verify before trusting -->`. Always-asked items with no safe default (Authorization, incidents, glossary meanings, CI, settings.json, new dependencies, compose/seed generation) are left as `TODO` under *Still open*.
- **Resume.** Run the command again. It reads `init-state.json`, says `Resuming at <layer>`, and never re-asks a recorded answer. If `scan.json` is stale (different `head`) or lacks baselines, the scan is redone first. The session hook reminds you: `harness setup in progress — resume with /harness-init`.
- **Workspace member.** From a workspace folder, `/agent-harness:harness-init <member>` (or `<member> one-shot`) runs the whole flow inside that member repo and touches nothing else. Without a member name in a workspace folder it stops, lists the members and prints that command. The agent cannot start `harness-init` itself, so a request like "create harness for portal" is answered with the command.
- **Existing harness.** Re-running init on a finished harness hands over to `harness-upgrade` (3.3a). A v1 repo it cannot bring up to date (`AGENTS.md`/`INVARIANTS.md` exist, no state file) gets the init flow, but the tree lists only the differences, and questions already answered by your files are skipped.

Updating the plugin and refreshing repo scripts

`claude plugin update agent-harness@agent-harness` (or `claude plugin marketplace update agent-harness` first) updates the plugin, its hooks and skills. The scripts init **copied into the repo** (`scripts/harness/cmd.sh`, `lint.sh`, `plan-check.sh`, `related-tests.sh`, `setup.sh`, `setup-proof.sh`, `sensor-proof.sh`, `local-guard.sh`, `sensors/no-prod-data.sh`, `sensors/lib/diffscan.py`, `.githooks/*`, `harness.mk`; in a workspace `scripts/harness/cmd.sh`, `plan-check.sh` and `ws.sh`) and the harness files themselves do not change by themselves. The session hook says `[agent-harness] harness scripts are older than the plugin (<n> files) — run /agent-harness:harness-upgrade`, and `harness-audit` lists them under observability. Run `/agent-harness:harness-upgrade` (3.3a), then commit the result and `scripts/harness/assets.json`.

A copy counts as unmodified when its sha256 matches `scripts/harness/assets.json` (written at install and refresh) or a released plugin copy, so repos set up before 2.0.2 (no manifest yet) are recognised too. To check by hand: `python3 <plugin>/scripts/assets.py status .`.

3.3 `/agent-harness:harness-audit`

Read-only. The only write is one line in `docs/harness/AUDIT-LOG.md`. Fixes go to `harness-garden`.

Mode	When	Does
`quick` (default)	monthly	`level.py --json --run-checks --log`; the 10 health questions (Integrity, Constraint propagation, Sensors, Coverage gaps, Enforcement gaps, Footgun freshness, Token budget, File size discipline, Content accuracy, Claude Code integration); count of `TODO`/`[verify]` markers. Recommends `full` if more than 2 questions fail.
`full`	quarterly, after an incident or major refactor	adds the 14-item completeness score (0–100), a content-quality pass, the simulated-task probe, `setup-proof.sh`, `sensor-proof.sh --recheck`, a comparison with the last log row; logs with `--health <score>`

Reading the report:

```
## Layers
guides        0.80  ✓ …
tools         0.50  ✗ no deny rules in .claude/settings.json
...
Level: Functional · target: Robust · blocked by: no permission rules in .claude/settings.json;
AGENTS.md has no Cadence section
## Fixes (ranked: blockers to the target first, then Critical gaps, then cheapest)
1. .claude/settings.json — add deny rules — closes blocker — layer tools
Health: 8/10 questions pass
Next: hand the fixes to /harness-garden
```

- **Layer scores** are the share of `level.py` criteria passing for that layer.
- **blocked by** lists the failing criteria of the *next* level. Fix those to move up.
- **Completeness bands:** 80–100 healthy · 60–79 gaps to schedule · below 60 address before the next major agent task · below 40 prioritise remediation.
- `setup-proof: SKIPPED (docker not available)` is reported as skipped, not as a pass.

3.3a `/agent-harness:harness-upgrade`

Brings a harness made by an older plugin version up to the installed one. Nothing is committed; you review and commit.

1. **Versions and scripts.** `migrate.py status` prints `plugin: recorded <x> · running <y>` (the recorded version is `"plugin"` in `scripts/harness/assets.json`). `assets.py install` replaces every **stale** (unmodified) plugin copy and adds new plugin scripts; a copy you **modified locally** is never overwritten and is listed with the `diff` command to compare it.
2. **Migrations.** Versioned and idempotent (registry: `skills/harness-upgrade/references/migrations.md`). Each is `auto` (new files, unedited plugin templates, local `.harness/` state: applied and listed), `propose` (edits to your files: `Makefile`, `AGENTS.md`, plans, `DECISIONS.md`, `.claude/settings.json`, `.agents/harness.json`; shown as **one diff**, written only after a single yes; `skip <id>` leaves one out) or `manual` (needs your decision, e.g. timing the test suite for `check-fast`). Active plans without a ticket get the key from the file or branch name, else a proposed `none (…)` the agent asks you to replace.
3. **Verify.** `harness-lint`, `check-fast` (sets `gate_wired` when it passes), `gate-proof.sh` (sets `gate_red_proven`), `level.py --run-checks --log` (one audit row), and the version is recorded.
4. **Report.** Before → after level, what was applied, what you accepted or declined, what still needs you, and the offer to run `harness-improve`.

Running it again changes nothing. In a workspace folder it upgrades the workspace files only and prints `/agent-harness:harness-upgrade <member>` for each member that is behind; `/agent-harness:harness-upgrade <member>` upgrades inside that member.

3.3b `/agent-harness:harness-improve`

Turns a thin harness into a useful one, from evidence only. init and audit end with "Run /agent-harness:harness-improve to strengthen <top 3 gaps>?" and, in an interactive session, ask whether to start now.

1. **Status.** Level and blockers, `TODO`/`[verify]` counts per file, invariants `not yet enforced`, skeletal sections (Agent Authorization, Footguns, Glossary, Context loading rows), risk paths without a module `AGENTS.md`, team rollout settings missing, stale scripts, last audit.
2. **Evidence.** Fix/revert/hotfix clusters in `git log` → footguns and invariants; risk paths → module guides; direction changes ("migrate to", dependency swaps) → ADR drafts; domain nouns not in `GLOSSARY.md` → glossary candidates; modules without a Context loading row; matches in the sensor catalog → sensors. No evidence, no suggestion.
3. **Interview.** One area per round: what it is for, `Found → Suggest` lines with their evidence, at most 5 numbered questions with options and a ➡️ default. "defaults" accepts the round.
4. **Apply and re-measure.** Each accepted item goes to its one owner file; machine-checkable rules go to `sensor-smith`, which proves the sensor on a fixture first. Then `harness-lint`, `level.py --run-checks --log`, and a before → after report.

3.4 `/agent-harness:harness-workspace`

For a plain folder holding several repos. It writes workspace files only and never writes into a member repo unless you name that repo ("create harness for portal"), in which case it runs `harness-init` there. See 04-workspace.md.

3.5 Everyday skills

Skill	Use it	What it does
`harness-verify`	Before "done", "fixed", "tests pass", a commit or a PR	Frames request, criteria and scope; runs `check-fast`, `check`, `smoke` (if relevant) through `cmd.sh`; dispatches `verifier`, `invariants-reviewer`, `scope-guard` in parallel; decides; prints `Verified: … Gate: … Criteria: … Red: … Reviewers: … Open: … Plan: …` and appends that block to the local `.harness/evidence/<plan file name>` (never committed). Max 3 fix rounds, then it hands back to you. When a failure repeated in the run, it offers `harness-learn` and runs it only on your yes.
`harness-session`	Session start, after `/clear` or compaction, before stopping	Orients from git and the active plan (matched by the branch's ticket key, e.g. `SHOP-12-pay-button`), shows the last local verification entry, checks health with `check-fast`, picks one step; on close appends a Progress log line, drafts a tracker comment for your yes, moves a finished plan to `done/` only when `plan-check.sh fresh` passes (latest local entry PASS, for the current code, nothing uncommitted), and offers `harness-learn` when the session shows a repeated failure (the stop gate blocked twice on it, a FAIL before the PASS, or your correction), running it only on your yes
`harness-learn`	After a repeated failure, a "don't do that again", or an incident; offered at close by `harness-session` and `harness-verify`	Picks the strongest fix: sensor → hook → check step → invariant → footgun → routing line → ADR → glossary. Changes to `INVARIANTS.md` or Authorization are proposed, not written.
`harness-garden`	Weekly, after big refactors, or with an audit's fix list	Lint + `harness-auditor`; fixes safe drift (dead links, ADR index rows, finished plans, duplicated rules, renamed commands); proposes the rest; may open `chore(harness): garden <date>` if Authorization allows
`harness-glossary`	New concept, two words for one thing, reviewer flags vocabulary	Adds a row after you choose the word and say what it means; checks a diff for "Avoid" words

Tickets, plans and where to see verification evidence

- **Tickets.** Every plan in `docs/plans/active/` carries `Ticket:` under its title: a Jira key (`SHOP-12`, also as a link or a list such as `SHOP-12, SHOP-13`; a key outside the project named in `docs/agents/issue-tracker.md` only warns), `#123`, an issue or browse URL, a `.scratch/...` issue file, or `none (<reason>)`. `harness-lint` fails on a missing or placeholder value; the workspace `make harness-lint` checks cross-repo plans the same way (default ticket: `issueKey` in `harness-workspace.json`). ADRs carry `Ticket:` too, and `docs/DECISIONS.md` has a Ticket column.
- **Naming.** Branch `<KEY>-<n>-<slug>`, plan `docs/plans/active/YYYY-MM-DD-<KEY>-<n>-<slug>.md`. The session hook finds the plan by the key in the branch name, then by the plan's `Ticket:` line, then by slug.
- **Where to see verification evidence.** Each harness-verify run that reaches a verdict appends `### <date> · <commit | uncommitted> · <PASS | FAIL | NEEDS-HUMAN>` plus Gate, Criteria, Red, Reviewers and Open lines to the **local** file `.harness/evidence/<plan file name>` (no plan: `<branch slug>.md`; a workspace keeps one block per repo in `<workspace>/.harness/evidence/`). `.harness/` is gitignored, so evidence is never committed. Others see it in the PR description and the tracker comment, which carry the full block plus the local path; you see it in the session hook's `last verification:` line. A plan is set to done only when `bash scripts/harness/plan-check.sh fresh .harness/evidence/<plan>` passes: the latest entry is PASS with a `Red: proven (…)` or `Red: n/a (<reason>)` line, its commit has the same code as HEAD (plan edits don't count), and no code is uncommitted. Lint does not look for evidence (CI and other clones have none); `harness-audit` says "evidence not available on this machine" instead of reporting a gap.
- **Tracker updates.** At close, harness-session (and harness-verify on "done") drafts a comment: a progress line mid-work, or the evidence block plus the plan path and next step when a step or the plan is

done, with a proposed transition. It posts only after you say yes, through a Jira/GitHub/GitLab connector or CLI in your session. It never stores tokens; without a connector it prints the text for you to paste.

Skills resolve commands with `scripts/harness/cmd.sh` in the repo, falling back to the plugin's own `harness-cmd.sh` when a (v1) repo has no copy.

3.6 Reviewer and builder agents

Agent	Role	Verdict / output
`verifier`	Runs the gate in a fresh context; checks each criterion has evidence; runs `red-proof.sh` (the change's tests must fail without it); looks for verification theatre	`PASS | FAIL`
`invariants-reviewer`	Diff vs `INVARIANTS.md`, module footguns, ADRs, glossary	`CLEAN | VIOLATIONS | NEEDS-HUMAN`
`scope-guard`	Every hunk traces to the request	`TIGHT | TRIM | SPLIT`
`cross-repo-reviewer`	Ownership, contracts, compatibility, merge order across workspace repos	`ALIGNED | MISMATCH | NEEDS-HUMAN`
`harness-auditor`	Drift and rot, layer by layer, using `level.py`	`HARNESS HEALTH: GREEN | AMBER | RED`
`repo-cartographer`	Maps the repo for init; writes only `.harness/scan.json`	report by layer
`sensor-smith`	Builds one sensor for one invariant, proves it with `sensor-proof.sh`; commits nothing	`SENSOR: … Proof: …`
`setup-smith`	Wires `setup-deps` / `setup-services` / `setup-migrate` / `seed` / `smoke`, proves with `setup-proof.sh`; compose/seed files only after a yes	`SETUP: … OK | FAIL (step) | SKIPPED`

The reviewers are read-only and did not write the change, so they are not grading their own work.

3.7 Dev environment: `make setup`, seed, reset, dev

`make setup` is safe to re-run. It works on a fresh machine and in any worktree. The report looks like:

```
setup: OK  (fresh clone · worktree feat-x · ports API=8123 DB=55432)
steps: deps ✓ · services ✓
data: migrate ✓ · seed ✓ · smoke ✓
needs you: STRIPE_KEY, docker login ghcr.io
```

The label is `fresh clone` when `.env` was just created from `.env.example`, `first run` before the first successful setup (marker `.harness/setup-ran`), else `re-run`. It exits non-zero only when a resolved step failed. Items under "needs you" alone exit 0.

Worktree isolation (default). In a linked worktree (`.git` is a file), setup derives `COMPOSE_PROJECT_NAME` from the repo and worktree folder names, picks free ports for every `*_PORT` in `.env.example`, suffixes `DB_NAME`, and writes them to `.env.local`. The main checkout keeps the ports from `.env.example` while they are free or held by its own compose project; a port something else holds gets a free one instead, recorded in `.env.local` (which wins on re-runs). Services start with `docker compose -f <file> --env-file .env --env-file .env.local -p <project> up -d` (Compose ≥ 2.17; with an older or unknown Compose version, `.env` and `.env.local` are merged into one temporary env file, later values winning). Set `HARNESS_SHARED_SERVICES=1`

(env or `.env.local`) to share one set of services instead. A repo's own `setup-services` target replaces that compose call. It runs without the picked ports in its environment, because Compose gives shell variables precedence over `--env-file`, so it reads them from `.env.local` or picks and writes its own there. `setup-migrate` and `seed` then see the values in `.env.local`.

Smoke in setup. Setup runs `smoke` last, and only when nothing is listed under "needs you". With `HARNESS_SMOKE` unset it reports `smoke n/a`, not ✓. For a server app whose smoke boots the app (and so needs the database configured first), set `HARNESS_SETUP_SMOKE = 0`: setup then reports `smoke skipped`, and agents check the running app with `dev-check` instead.

Checking a running app: `make dev-check`. Set `HARNESS_HEALTH` above the include, as a URL (`http://localhost:8080/health`) or a command that exits 0 once the app is up. `make dev-check RUN='curl -fsS localhost:8080/api/orders'` starts `make dev` in the background, waits up to `HARNESS_HEALTH_SECS` (120), runs `RUN`, and always stops the app (also when health or `RUN` fails; the log is `.harness/dev-check.log`). `make dev-stop` stops a leftover one. Once `dev-check` exists, the Bash guard refuses an agent's long-lived `make dev` (also as `nohup make dev &`, `bash -c 'make dev'`, `FOO=1 make dev`) and any word in `HARNESS_DEV_BLOCK` (`HARNESS_DEV_BLOCK = run-local.sh spring-boot:run`). Your own terminal and `!` commands are not affected.

seed / reset / dev exist only when you set `HARNESS_SEED`, `HARNESS_RESET` or `HARNESS_DEV` above `include harness.mk`. Unconfigured, `make seed` says "No rule" and `make setup` skips the step. Before running, `seed` and `reset` call `local-guard.sh`, which refuses any non-local database host.

3.8 Using a runner other than make

Pick "register another" in the Tools section, or write `.agents/harness.json` yourself (see 02-file-reference.md). Hooks, the verifier, the audit, git hooks and the workspace gate all resolve commands through `cmd.sh`, which reads this file first and falls back to `make <target>`. Use `{T}` unquoted for the test path. `AGENTS.md` Verification must name the same runner commands, or `harness-lint` fails. Keep it in the short form (done, fast loop, all commands) and name the runner's list command, recorded as optional `"help"` in the file (`just --list`, `npm run`, `task --list`).

3.9 Where checks run, and why

Place	Default	Notes
Agent hooks (stop gate, guards)	on	Agents only; Claude Code
Git hooks: pre-commit runs `check-fast`, pre-push runs `check`	on	With husky, lefthook or pre-commit, the manager's config calls `.githooks/*`. Otherwise `core.hooksPath .githooks`, set by `make setup`. The session hook warns `git hooks not installed — run: make setup`.
CI workflow	opt-in	Generated only on a yes. It is the only check nobody can skip, and it is required for Enterprise.

The stop gate runs `check-fast` only when source changed (uncommitted files outside `docs/`, `*.md` and `.harness/`, or commits ahead of the base) and the tree differs from the last green run. A passing run is silent. After a block, the next stop re-runs `check-fast` only if the code changed since that block, up to 3 blocks in a row; then the stop is allowed and you see `agent-harness: stop gate gave up (…); check-fast still failing in: <repo>`. Nothing is written into a repo without `AGENTS.md`.

Parallel implementers. The same gate runs when a subagent finishes, but only for a subagent working in its own linked git worktree (for example, implementers a spec-driven flow starts one per ticket). Claude Code (2.1.x) records a block on a subagent's stop but doesn't keep the subagent working, so the gate enforces at the merge instead: when `check-fast` fails in that worktree, the branch is recorded as red in `.git/harness-red/<branch>` (shared by all worktrees), the subagent's transcript gets the failures and a hint to run the `setup` target if dependencies are missing, and the Bash guard denies `git merge`, `git cherry-pick` or `git pull .` of that branch. The marker clears when a later stop in that worktree passes (for example, after the orchestrator sends the failures back to the implementer); after fixing it some other way, run `check-fast` there and delete the marker as the deny message says. Git hooks still refuse red commits in every worktree. Reviewers and explorers working in the main checkout are never blocked by the main session's unfinished work. `AGENTS.md` → Agent skills rule 8 covers the rest of such a flow: one plan for the spec with every ticket key in `Ticket:`, worktrees under `.worktrees/`, harness-verify on the integration branch after the final review, and ticket transitions only after a yes. Existing repos get rule 8 from `harness-upgrade` (migration `agents-parallel`).

3.10 Working alongside other skill packs

The harness holds the facts; packs bring process. Any pack works inside it if it follows `AGENTS.md` → `## Agent skills`:

- `INVARIANTS.md` and `AGENTS.md` outrank any skill.
- Specs, plans, ADRs and notes go where Work Artifacts says, not in a pack's default folder.
- Proof comes from the resolved `check`; `harness-verify` runs last, whatever process ran before it.
- Run one process skill per phase (not two planning or TDD flows over the same work).
- Never write to `CLAUDE.md`. Setup blocks go under `## Agent skills`, guidelines under Working Principles. The guards enforce this.
- Never loosen hooks, guards, permissions or checks.

Scope: pin only `agent-harness` in project settings; keep other packs in user scope (`~/.claude/settings.json`) or local scope (`.claude/settings.local.json`). Before sharing a pack with the team, read its source, check its always-on token cost (`claude plugin details <name>`), list its hooks and MCP servers, and try it on one repo for a week. When a pack's default conflicts with the repo, add one line to `AGENTS.md`. Don't fork the pack.

3.11 Settings and environment variables

Variable	Effect
`HARNESS_STOP_GATE=0`	Turns off the stop gate (and the red-branch check for worktree subagents) for the session
`HARNESS_GUARD=0`	Turns off the Bash and file guards for the session (not recommended)
`HARNESS_SHARED_SERVICES=1`	`make setup` shares local services instead of an isolated compose project and ports (env or `.env.local`)
`HARNESS_ALLOW_UNVERIFIED_DONE=1`	The pre-commit hook lets a plan close when this machine has no `.harness/evidence/` at all (with a notice); stale or failing evidence still blocks
`HARNESS_ALLOW_SEED=1`	Lets `seed/reset` run when `local-guard.sh` is missing, and overrides its non-local refusal (with a warning)
`HARNESS_BASE`	Base ref. As an env var it overrides the stop gate's base; as a Makefile variable it sets `check-fast`'s changed-files base. Default: `origin/HEAD`, else `origin/main`, else `origin/master`.
`HARNESS_PROOF_KEEP_ENV=1`	`setup-proof.sh` copies your `.env.local` into the clone

Variable	Effect
`HARNESS_DEFINE_<NAME>=0`	Makefile: don't define that `harness.mk` target (yours wins)

3.12 Troubleshooting and FAQ

"Stop blocked by agent-harness: a repo's check-fast failed on your changes." The agent changed code and `check-fast` failed. Fix the failures shown. Don't disable checks. If you must stop anyway, set `HARNESS_STOP_GATE=0` for that session. In a workspace, the block also lists `Not gated (no complete harness or gate not wired), changed anyway:` members.

`make test T=<a failing test>` exits 0. The test command discards its exit code (a `HARNESS_TEST_*` value ending in `; :`, added to swallow the appended T) or passes when 0 tests ran (`-Dsurefire.failIfNoSpecifiedTests=false`, `--passWithNoTests`, a JUnit 4 class without the vintage engine). harness-lint warns about `; :`; `bash scripts/harness/gate-proof.sh` shows which target stays green. Put `{T}` / `{CHANGED}` where the argument goes, or end in `&& :`, and add the 0-tests check for your stack (`layout.md`, **Test commands keep their exit code**).

`sensor: cannot resolve HARNESS_BASE origin/main; fetch it or set HARNESS_BASE.` A sensor that checks only new code needs the base branch and its merge-base with HEAD. A shallow or single-branch clone (common in CI) or a never-fetched remote lacks one. Run `git fetch origin main` (with `--unshallow` for a shallow clone), or set `HARNESS_BASE` to a ref you have. The sensor fails instead of passing because otherwise it would check nothing.

`make setup` with your own `setup-services`: "port is already allocated", or the app can't reach a service. Before 2.0.10, `setup.sh` exported the `*_PORT` values it picked, and Compose prefers shell variables over `--env-file`, so the exported port beat the one your target chose. Run `/agent-harness:harness-upgrade` to refresh `setup.sh`. Your target should read ports from `.env.local`, or write the ones it picks there.

The stop gate or a git hook says nothing during setup. That's intended. While `.harness/init-state.json` exists without `"gate_wired": true`, the stop gate is silent and git hooks print `harness: setup in progress, skipping`. Finish or resume `/agent-harness:harness-init`.

`[agent-harness] gate not wired — <reason>; run /agent-harness:harness-init to finish wiring.` Init finished every layer but `check-fast` never passed, so the stop gate and git hooks are off. Re-run `/agent-harness:harness-init`: a tool the environment can't run is wired as `@echo "<tool>: skipped (<why>, <date>)"` with a `not yet enforced` invariant, so the rest of the gate can wire.

`[agent-harness] git hooks not installed — run: make setup.` `.githooks/` exists, `core.hooksPath` is unset, and there's no hook manager. Run `make setup` or `git config core.hooksPath .githooks` (the agent may run that exact command). Reading it (`git config core.hooksPath`, `git config --get core.hooksPath`) is allowed. The guard denies any other `core.hooksPath` value, `--unset`, `--global` and `-c core.hooksPath=` overrides.

A git hook prints `harness: check-fast not wired, skipping.` `cmd.sh` couldn't resolve the target (`make -n check-fast` failed, or it's not in `.agents/harness.json`). Run `make -n check-fast` to see why.

`harness-lint` failures. Each line names the file and the fix. Common ones:

- `CLAUDE.md has real content` → move it to `AGENTS.md`.
- `AGENTS.md names 'make x' but the Makefile has no 'x' target` → fix the name or add the target.
- `INVARIANTS.md bullet lacks '— Severity: … — Enforced by: …'` → add both.

- `docs/adr/X.md is not listed in docs/DECISIONS.md` → add the row.
- `sensor recheck failed` → a sensor no longer fires on its fixture, or now fails on clean code. Fix the sensor, never the fixture.
- TODO/`[verify]` lines are warnings, not failures.

`lint: not configured` / `test: not configured` fails `check`. The variable was never wired. Set it to the real command, or decline the tool: `HARNESS_LINT = @echo "lint: none (declined <date>)"` plus a `not yet enforced (declined <date>)` invariant.

Seed refused. `seed: scripts/harness/local-guard.sh missing — refusing …` → reinstall the script, or set `HARNESS_ALLOW_SEED=1`. `local-guard: refusing, non-local target: DB_HOST=db.prod.example.com …` → point the variable at a local service; override with `HARNESS_ALLOW_SEED=1` only if you're sure. `make: *** No rule to make target 'seed'` → `HARNESS_SEED` isn't set above the include.

Level stuck. Run `/agent-harness:harness-audit` and read `blocked by:`. Common causes: `check not passing (no .harness/last-check pass; run with --run-checks)`; `no automatic enforcement` (run `make setup`, or add check-fast to your hook manager); `no invariant is machine-enforced`; a Security/Data Integrity rule still `not yet enforced` (Robust); `AUDIT-LOG.md needs 2+ rows` (Enterprise needs history, so it can't be reached on day one). The level never exceeds what the criteria measure, whatever the target says.

`CLAUDE.md` write blocked. That's intended: `CLAUDE.md files are shims (@-references only). Put instructions in AGENTS.md — …`. Move the content to `AGENTS.md` → `## Agent skills` or Working Principles. A pack installer that insists on `CLAUDE.md` should be pointed at `AGENTS.md`.

Writing `.env` blocked. `Writing to .env is blocked: secrets files are human-managed. Update .env.example instead.` Applies to Edit/Write and to Bash writes (`>`, `>>`, `tee`, `cp`/`mv`, `curl -o`, `sed -i`) into `.env` or `.env.*`; `.env.example`, `.env.sample` and `.env.template` stay writable. Edit it yourself.

A Bash command was denied inside `bash -c`, `sh -c` or `eval`. The guard checks those payloads like any other command, and an abbreviated git option (`--no-veri`, `--forc`) like the full one.

Edit to `INVARIANTS.md` or `harness-workspace.json` asks for confirmation. These need human sign-off. Approve only if you asked for the change. Bash writes into `INVARIANTS.md` (`>`, `>>`, `tee`, `cp`/`mv`, `sed -i`) ask the same way.

Commit refused: `<plan> is closed as done without a fresh PASS`. The pre-commit hook checks every plan the commit closes (moved to `done/` or set to `Status: done`) with `plan-check.sh fresh`. Run harness-verify, commit the code, then close the plan in its own commit: plan-only commits keep the evidence fresh. `the PASS entry has no red proof`: the entry was written before 2.0.8, or the verifier could not record a red proof; run harness-verify again.

Verify reports `Red: not proven`. The change's tests pass without the production change, so they don't prove it: write a test that fails first, then make it pass. A pure refactor or config change with no new behaviour is recorded as `Red: n/a (<reason>)` by the verifier. `Red: inconclusive` means the tests could not run in a scratch worktree (missing dependencies or services); the verifier tries by hand, else verify stops as NEEDS-HUMAN.

Push to main asks. `Pushing to <branch>. AGENTS.md Agent Authorization normally requires a human for this.` Force-push, `git reset --hard`, `git clean -f`, `git checkout .`, `git branch -D`, `git stash clear`, `--no-verify`, `HUSKY=0`/`SKIP=…` before git, and recursive `rm` of `/`, `~`, `.`, `..`, `*` are denied. `DROP/TRUNCATE TABLE|DATABASE|SCHEMA`, `kubectl delete`, `terraform destroy`, `terraform apply -auto-approve` and `helm uninstall` ask.

`.claude/settings.json` write refused. Claude Code may refuse it. The plugin shows the full JSON; add it yourself. It does not work around the refusal.

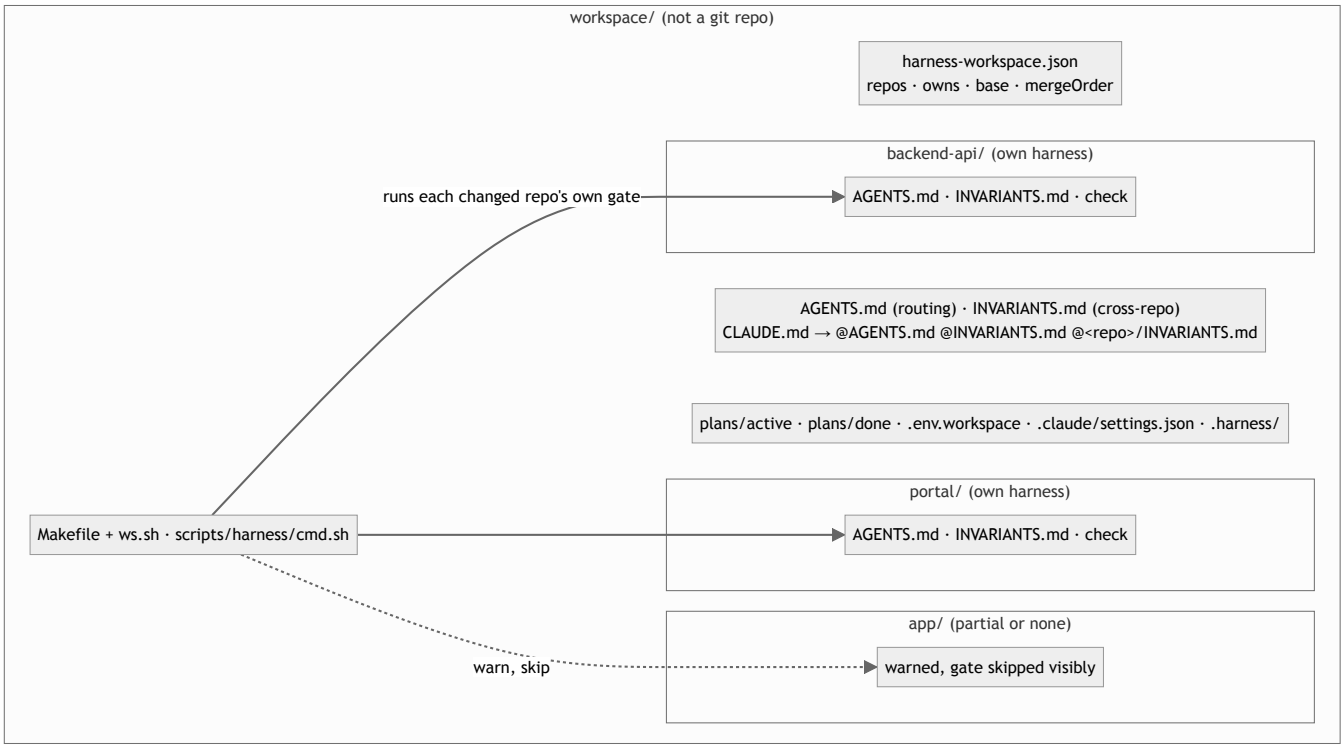
`setup-proof: FAIL (setup not wired …).` The clone has no resolvable `setup`. Commit the `Makefile`/`harness.mk` wiring first; the proof only sees committed files.

Session keeps saying `harness setup in progress`. It shows until `init-state.json` records all five layer keys. With a Functional target only two layers are interviewed; run `/agent-harness:harness-init` again to finish the rest (or raise the target), or delete `.harness/init-state.json` once `gate_wired` is true and you're done. Deleting it before the gate is wired loses resume state.

4. Multi-repo workspaces

Use this when one feature spans several repos, for example a backend API, a web portal and a mobile app. A **workspace** is a plain folder that is **not** a git repo and holds those repos side by side. Each repo keeps its own harness. The workspace adds only what spans repos: which repo owns which change, contract-first rules, one gate that runs every changed repo's own gate, and cross-repo plans.

Layout



Harness state per member

State	Meaning	Effect
present	`AGENTS.md` + `INVARIANTS.md` + a resolvable `check` command	gated by its own `check` / `check-fast`
partial	some of those	**warning**, gate skipped visibly
none	none of those	**warning**, gate skipped visibly

A partial or missing harness is never a failure, and the workspace never writes into the repo to fix it.

Setting it up: `/agent-harness:harness-workspace`

Run it in the workspace folder.

1. **Find the repos.** It finds immediate subfolders that are git repos (a worktree's `.git` file counts) and records each one's remote, default branch, harness state and what it owns (from its `AGENTS.md`). A repo that isn't checked out, or has no remote, produces a warning.

2. **Confirm** one table with you: repo · path · harness · owns · merge order. It asks what isn't clear, especially which repo owns the shared contract.
3. **Write workspace files** after one yes: `harness-workspace.json`, `AGENTS.md`, `INVARIANTS.md`, `scripts/harness/cmd.sh`, `Makefile`, `ws.sh`, `CLAUDE.md` (via `make claude-md`), `plans/`, `.claude/settings.json` (shown for you to add if Claude Code refuses the write).
4. **Local wiring.** It reads the backend's port from `<backend>/.env.local` (written by that repo's `make setup`; if missing, it says to run `make setup` first and leaves a `TODO`) and writes `.env.workspace`, e.g. `API_BASE_URL=http://localhost:<port>`. For each frontend or app, it names the member env var that consumes the value (with `file:line`, or `[verify]`) and tells you to copy it there. It never writes a member's `.env` or `.env.local`.
5. **Cross-repo sensors (optional).** Read-only checks such as "backend OpenAPI spec matches the frontend client types", wired into workspace `check`.
6. **Validate** with `make harness-lint` and `make status`.

Never writes inside a member repo. To give a member its own harness, run `/agent-harness:harness-init <member>` from the workspace folder (`<member> one-shot` skips the interview); it works inside that member only. Asked "create harness for portal", the agent answers with that command, since it cannot start `harness-init` itself.

After a plugin update run `/agent-harness:harness-upgrade` in the workspace folder: it refreshes `ws.sh` and `scripts/harness/`, migrates the workspace files (plan template, cross-repo plans, `AGENTS.md`) and prints `/agent-harness:harness-upgrade <member>` for each member whose harness is behind; it never writes into a member itself. Verification evidence for cross-repo plans is local: `.harness/evidence/<plan file name>`, one block per repo.

Workspace commands

Command	Does
`make status`	Per repo: branch, uncommitted files, commits ahead of base, harness state
`make changed`	Repos with uncommitted changes or commits ahead of base
`make check`	`check` in every changed, present repo (the done gate). Prints `ok`, `warn` or `FAIL` per repo.
`make check-fast`	Same, with `check-fast`
`make test R=<repo> [T=<path>]`	One repo's tests
`make branch B=<name> R="<repo> <repo>"`	Create or switch the same branch in the named repos only
`make setup`	Each present repo's `setup`, in `mergeOrder`
`make bootstrap`	Clone any manifest repo that is missing
`make claude-md`	Rewrite `CLAUDE.md` with member `@<repo>/INVARIANTS.md` imports
`make harness-lint`	Workspace map consistent; every member in `AGENTS.md`; `mergeOrder` names known repos

Base per member: `origin/<base>` from the manifest; if none, `origin/HEAD`, else `origin/main`, else `origin/master`. A present member with no `check-fast` (or `setup`) target prints `warn <name>: no 'check-fast' command — skipped`.

Token budget. If the workspace `AGENTS.md`, `INVARIANTS.md` and the present members' `INVARIANTS.md` together exceed 5,000 tokens (characters / 4), `make claude-md` drops the member imports, appends missing

links under `## Member invariants` in the workspace `AGENTS.md`, and warns `member invariants exceed budget — linked from AGENTS.md (## Member invariants) instead`.

Cross-repo rules (workspace `INVARIANTS.md`)

- Change the contract first: the repo that owns an API changes and tests it before any client — Critical, enforced by `cross-repo-reviewer`.
- Merge in `mergeOrder`, providers before clients — Critical, enforced by review.
- No breaking API change without a version: additive only until every client has shipped — Critical, enforced by `cross-repo-reviewer`.
- Never write one repo's code in another or import across repos by path — Important.
- One branch name and one issue key across the repos a feature touches — Nice-to-have, visible in `make status`.

Inside a repo, that repo's `AGENTS.md` and `INVARIANTS.md` also apply. If they conflict with the workspace rules, the stricter rule wins; if that is unclear, the agent stops and asks.

How it behaves at run time

- **Session start** (agent opened in the workspace folder): `[agent-harness] workspace: <name> — each change goes to the repo that owns it …`, then one line per member (`<name>: <branch> · <n> uncommitted · harness <state>` or `not checked out`), the active cross-repo plan, and a reminder of the workspace gate.
- **Stop gate:** runs `check-fast` in each changed, present member with that member's own wiring. A failure blocks the stop and names the repo. Changed members without a complete harness, or whose gate is not wired yet (`gate_wired` not true, or `check-fast` unresolvable), are listed under `Not gated (no complete harness or gate not wired), changed anyway:` when the stop is blocked. The session view marks such a member `harness present (gate not wired)`.
- **`harness-verify` (workspace mode):** `make changed`, then gate and reviewers per changed repo with that repo's diff and rules. When more than one repo changed, it adds `cross-repo-reviewer` (ownership, contract consistency, backward compatibility, merge order). Done only when every repo passes and the verdict is `ALIGNED`.
- **Guards apply everywhere:** force-push, hook bypass and pushes to a repo's default branch are caught even via `cd <repo> && git …` or `git -C <repo> …`.
- **`level.py`** in a workspace also measures each member. For Enterprise, every member must be at Functional or higher.

Notes

- The workspace folder can stay unversioned. To share it, keep the workspace files in a small git repo whose `.gitignore` lists every member path, and clone members with `make bootstrap`.
- A member repo's `.claude/settings.json` does not apply when Claude Code starts in the workspace folder; the workspace's own `.claude/settings.json` does. Keep both pinned to the harness plugin.
- Workspace files (`AGENTS.md`, `Makefile`, `plans/`, `.harness/`) are never copied or committed into a member repo.